

第9章 组合逻辑电路设计

第一节 组合逻辑电路简介

数字逻辑电路分为组合逻辑电路（以下简称“组合电路”）和时序逻辑电路（以下简称“时序电路”）。组合逻辑电路由各种逻辑门构成, 电路在任何时刻的输出都由其当前的输入信号决定, 其逻辑功能可由一组布尔函数确定。时序逻辑电路由逻辑门和存储器构成, 其输出是当前输入和存储器状态的函数, 而存储器状态是历史输入的函数。因此, 时序电路的输出不仅依赖于其当前输入, 也依赖于其过去的输入, 其逻辑功能由输入和内部状态共同确定。

组合逻辑电路由相互连接的逻辑门构成, 逻辑门接收输入信号后产生输出信号。这个过程是将输入的二进制数据信息转换成需要的输出数据。图 9.1 所示是组合逻辑电路的框图, 其中 n 个二进制变量来自于外部信源, m 个输出变量送给外部接收者。每个输入和输出变量实质上对应于二进制信号的逻辑 1 和逻辑 0。在许多应用中, 信源和信宿都是寄存器, 如果组合电路中包含寄存器, 那么整个电路就要按时序电路对待。



图 9.1 组合逻辑电路框图

n 个输入变量有 2^n 种输入组合。对于每一种组合, 只有一种可能的输出值与其对应。把每种输入组合和与其对应的输出值列成真值表, 这样, 组合电路就能使用真值表来详细描述, 也可由 m 个布尔函数来描述, 一个函数对应一个输出变量, 每个输出是 n 个输入变量的函数。在第 2 章中, 我们学习了表达离散信息量的二进制数。二进制变量用来表示物理上的电压或其他类型信号。信号可以由数字逻辑门控制以实现特定功能。第 2 章介绍了布尔代数作为从代数上表达逻辑函数的手段, 并学习了怎样化简布尔函数以得到简单的门电路实现。本章的目的是运用前面学到的知识, 形成组合电路的系统分析和设计步骤。一些典型例子的解法将会给理解数字系统提供有用的重要基本函数样本。这里强调两项工作: (1) 分析给定逻辑电路的功能; (2) 综合实现给定功能的电路。

设计一个数字系统往往需要好几个组合电路。这些电路已经能集成为基本单元, 它们通常用来完成数字系统设计中的特定功能。在本章中, 我们介绍一些最重要的基本组合电路单元, 如加法器、减法器、比较器、译码器、编码器等。这些器件在集成电路中

被归类为 MSI（中规模集成）电路，也作为标准单元用于复杂的 VLSI 电路，如专用集成电路（ASIC）。标准单元函数在 VLSI 内部的相互连接方法与其在多芯片的 MSI 电路设计中相同。

第二节 组合逻辑电路分析

组合电路的分析就是确定组合电路的实现功能。通过分析给定的逻辑图，得到一组布尔函数、真值表或电路功能的说明。如果待分析的逻辑图具有函数名或者功能实现的说明，那么分析问题就简化为对所述功能的验证。分析的执行可通过人工求出布尔函数或真值表，也可利用计算机仿真程序实现。

分析的第一步是确认给定电路是组合电路而非时序电路。组合电路只有逻辑门，没有反馈路径或存储元件。所谓反馈路径是指将第一个门的输出连接到第二个门的输入，同时第二个门的输出连接到第一个门的输入。存在反馈路径的数字电路称为时序电路，其分析有其他方法，这里不予考虑。

一旦逻辑图被确认为是组合电路，就可以进一步得到其输出布尔函数或者真值表。如果该电路的功能还没有得到确定，那么就需要从布尔函数或真值表来解释该电路的功能。这个工作需要经验的积累，并对各种数字电路非常熟悉。

从逻辑图得到输出布尔函数的基本步骤如下：

1. 将所有输入变量函数的逻辑门输出用任意符号标出。确定每个门输出的布尔函数。
2. 标出输入变量函数的逻辑门，前面标注过的逻辑门采用其他任意符号。求出这些逻辑门的布尔函数。
3. 重复第 2 步的操作，直到求得电路的输出。
4. 把前面的函数重复代入，得到仅有输入变量的输出布尔函数。

以图 9.2 所示的组合电路为例来说明上述基本分析步骤。该电路有三个二进制输入变量 A 、 B 和 C ，两个二进制输出 F_1 和 F_2 。各个门的输出用中间符号标出：仅有输入变量函数的门输出有 T_1 和 T_2 ，输出 F_2 容易从输入推导出来。三个输出的布尔函数为：

$$F_2 = AB + AC + BC$$

$$T_1 = A + B + C$$

$$T_2 = ABC$$

接下来，把门的输出看作已定义符号的函数：

$$T_3 = F_2' T_1$$

$$F_1 = T_3 + T_2$$

为了把 F_1 表示成 A 、 B 和 C 的函数，需要做以下一系列代换：

$$F_1 = T_3 + T_2 = F_2' T_1 + ABC = (AB + AC + BC)'(A + B + C) + ABC$$

$$= (A' + B')(A' + C')(B' + C')(A + B + C) + ABC$$

$$= (A' + B'C')(AB' + AC' + BC' + B'C) + ABC$$

$$= A'BC' + A'B'C + AB'C' + ABC$$

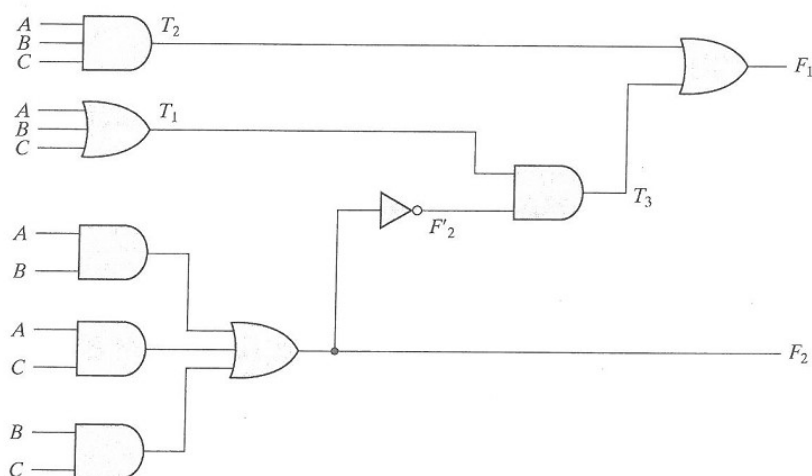


图 9.2 逻辑图分析实例

如果想进一步研究和确定这个电路实现的信息传输任务，可以根据布尔表达式画出电路图并且设法识别常见的运算。 F_1 和 F_2 的布尔函数实现电路相当于一个全加器电路，将在后面进一步讨论。

若输出布尔函数已知，依据真值表可以直接推导出电路。也可以省略布尔函数的推导，直接从逻辑电路图得到真值表，步骤如下：

1. 确定电路中输入变量的个数 n ，建立 2^n 种可能的输入组合，并把这些从 0 到 2^n-1 之间的二进制数列在一张表上。
2. 把选中门的输出用任意符号标示出来。
3. 得到那些仅有输入变量函数的门的输出真值表。
4. 继续求取先前定义过函数值的门的输出真值表，直到所有输出都确定为止。

仍用图 9.2 所示电路来举例说明这个过程。在表 9.1 中，对应于三个输入变量形成了八种可能的输出组合。 F_2 的真值表由 A 、 B 和 C 的值直接决定，有两个或三个输入等于 1 时， F_2 的值为 1。 F_2' 的真值表是对 F_2 的求反。 T_2 和 T_1 的真值表是输入变量的与、或函数。 T_3 的值由 T_1 和 F_2' 推出：当 T_3 和 F_2' 都等于 1 时， T_3 等于 1；否则 T_3 等于 0。

最后， F_1 等于 1 的条件是 T_2 和 T_3 至少有一个等于 1。由真值表可得， A 、 B 、 C 、 F_1 和 F_2 组合分别与后面介绍的全加器真值表中的 x 、 y 、 z 、 S 和 C 相同。

表 9.1 图 9.2 逻辑图的真值表

A	B	C	F_2	F'_2	T_1	T_2	T_3	F_1
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

另外一种分析组合电路的方法是逻辑仿真。由于能够产生有意义输出的输入数可能非常巨大，仿真没有实际意义。但是，它在电路的功能与特性的验证方面有非常实用的价值。

第三节 组合逻辑电路设计步骤

组合电路设计根据设计目标要求，画出逻辑电路图或得到逻辑图的一组布尔函数。其步骤如下：

1. 根据电路要求，确定需要的输入数和输出数，并给每个变量分配一个符号。
2. 列出输入和输出之间关系的真值表。
3. 求输出函数的最简表达式。
4. 画出逻辑电路图，并验证设计的正确性。

真值表由输入栏和输出栏构成，输入栏包含对应于 n 个变量的 2^n 个二进制数，输出的二进制值则根据设计要求确定。真值表中定义的输出函数对组合电路做了准确描述。将设计要求正确翻译成真值表是很重要的一步，设计要求通常是不完全的，任何错误的翻译都可能导致不正确的真值表。

对真值表所列出的输出二进制函数要设法进行化简，例如代数化简法、卡诺图法或计算机化简程序等。因此，有多种不同的简化表达式可供选择。在特殊用途中，会有特殊的选择标准。在实际设计过程中，必须考虑一些约束条件，如门的个数、门的输入数、信号通过门的传播时延、连线数、每个门的驱动能力以及在集成电路设计中要考虑的各种其他标准。由于每个约束的重要程度取决于具体应用，因此很难对一个可行的电路方案进行概述。大多数情况下，首先是为满足基本目标而进行化简，例如把化简后的布尔函数表示成标准式，然后采用其他措施进一步化简，以满足其他性能标准。

码变换举例

同一个离散单元信息可以用许多不同的代码表示，这就导致了不同数字系统用的是不同的代码。有时需要将系统的输出作为另一个系统的输入，如果这两个系统用不同的代码表示相同的信息，那么就需要在两个系统间加入变换电路。因此，码变换器是使两个系统协调工作的电路，即使这两个系统使用不同的二进制码。

将二进制码 A 变换为二进制码 B ，输入线路必须要支持代码 A 所用到的元素组合，而输出线路必须能够产生对应的代码 B 的组合。组合电路通过逻辑门来实现这个转换。

一个 n 位二进制码就是一个 n 位的比特组合，它有 2^n 种不同的 0、1 组合，每种组合都代表一个编码。一个四元素集可以用两个比特来编码，每个元素被赋予四种比特组合中的一个：00、01、10、11。八元素集需要三个比特，而十六元素集则需要四个比特。一个 n 位码所表示的二进制数从 0 到 2^n-1 。每个元素必须分配唯一的比特组合，两个元素不能有相同的编码值，否则码的分配就会产生混乱。如果要编码的元素个数不是 2 的幂次，则二进制编码会有一些码组无处分配，10 个十进制数集就是如此。区分 10 个元素的二进制码至少要有 4 位，而 4 位二进制码可以有 16 种不同的组合。因此，其中有 6 种编码没有被分配。将 4 位排列为 10 种不同的组合就能得到不同的二进制码。表 9.2 列出了两种二进制码，其中一种是十进制数最常用的码对应的二进制数分配，也就是二进制编码十进制数（二-十进制码），通常简称为 **BCD 码**。另一种是余 3 码，其每个码由对应的二进制数值加上 3 而来。

表 9.2 十进制数的两种不同的二进制码

十进制符	BCD 码	余 3 码
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

下面通过把二-十进制（BCD）码转化为余 3 码的例子来说明前述的设计步骤。

由于两种码都采用四位二进制表示一个十进制数，就必然存在四个输入变量和四个输出变量。四个输入变量记为 A 、 B 、 C 、 D ，四个输出变量记为 w 、 x 、 y 、 z 。输入和输出真值表见表 9.3。

输入码组和它们对应的输出可从表 9.2 直接得到。四个二进制变量有十六种组合，但真值表中只列出了十个，没被列出的六个输入变量的组合为“无关项”，这些值在 BCD 码中无意义，我们假设它们在实际电路中不会出现。因此，可以给输出变量随意分配 1 或 0，从而得到更简单的电路。

表 9.3 BCD 码变换举例的真值表

输入 BCD 码				输出余 3 码			
<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>w</i>	<i>x</i>	<i>y</i>	<i>z</i>
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

对图 9.3 中的卡诺图画圈，可以得到输出布尔函数的最简式。每个卡诺图与电路的四个输出之一相对应，而电路是四个输入变量的函数。标 1 的方格对应输出为 1 的最小项，它们可以由真值表中的输出栏得到。例如，输出 *z* 有五个 1，因此，*z* 的卡诺图就有五个 1，每个对应一个使 *z* 等于 1 的最小项。10 到 15 这六个无关的最小项用 X 表示。每个卡诺图下面列了输出函数积之和式的最简表达式。

从化简后的布尔表达式可以直接得到每个输出的两级门电路逻辑图。可能还有一些其他的逻辑图用来实现这个电路。为了用普通的逻辑门得到两个或更多的输出，可以对图 9.3 中的表达式进行代数变换。这个操作如下所示，用以说明三级以上门电路构成多输出系统的灵活性：

$$\begin{aligned}
 z &= D' \\
 y &= CD + C'D' = CD + (C + D)' \\
 x &= B'C + B'D + BC'D' = B'(C + D) + BC'D' \\
 &= B'(C + D) + B(C + D)' \\
 w &= A + BC + BD = A + B(C + D)
 \end{aligned}$$

这些表达式的实现电路如图 9.4 所示。注意，输出为 $C + D$ 的或门对三个输出函数都起作用。

不计输入的非门，积之和式的实现需要 7 个与门和 3 个或门。图 9.4 所示电路用了 4 个与门、4 个或门和 1 个非门。

如果输入只能为原变量，则第一种实现就需要对变量 *B*、*C* 和 *D* 进行取反，第二种实现则需要对变量 *B* 和 *D* 取反。

这样， 三级逻辑电路就需要更多的逻辑门， 所有逻辑门的输入不超过两个。

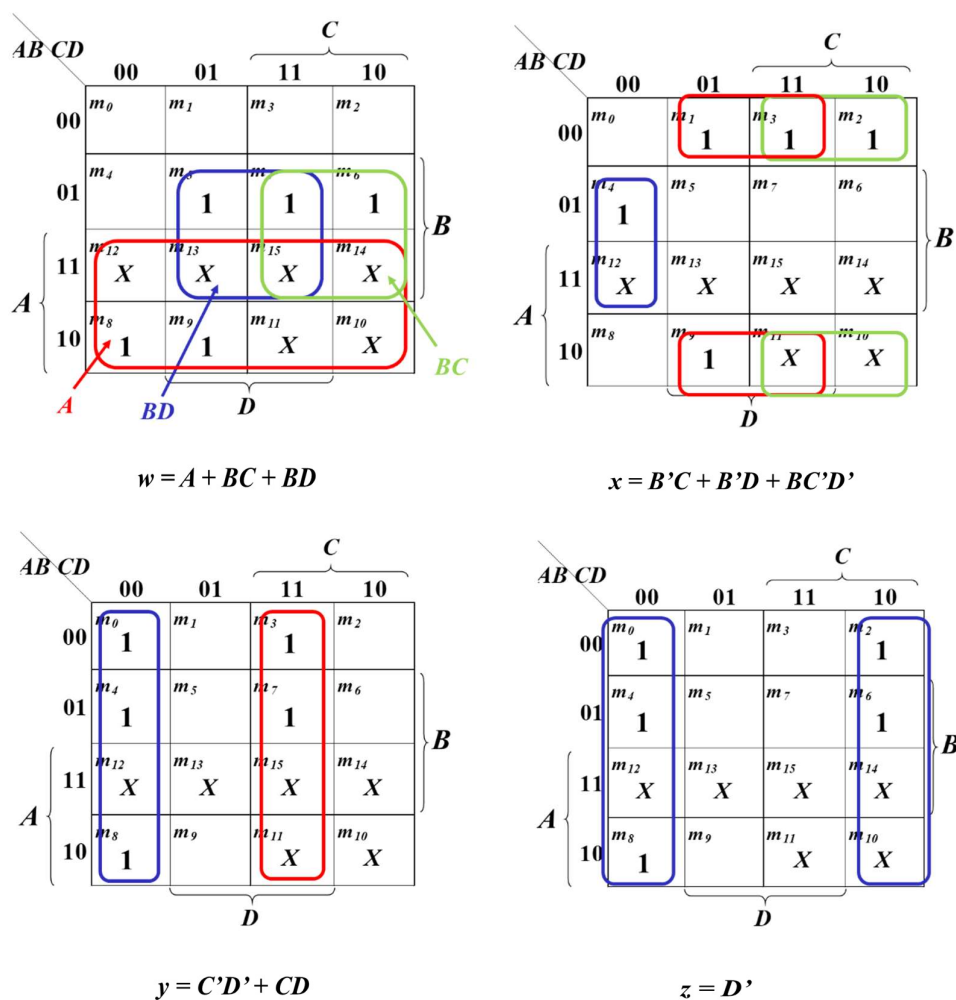


图 9.3 BCD 码到余 3 码转换电路的卡诺图

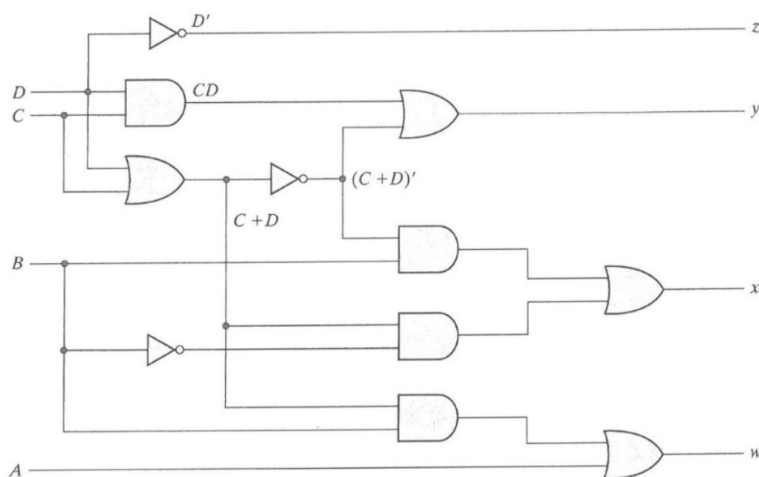


图 9.4 BCD 码到余 3 码转换器的逻辑

第四节 数值比较器

两个数的比较就是确定一个数是大于、小于还是等于另一个数，数值比较器是对两个数 A 和 B 进行比较，以确定它们相对大小的组合电路，比较结果是由三个二进制变量区分 $A > B$ 、 $A = B$ 和 $A < B$ 三种情况。

一方面，两个 n 位数比较电路的真值表有 2^{2n} 项，即使当 $n=3$ 时也十分烦琐。另一方面，比较器电路有一定的规律性，那么一个固有的、有明确规律的数学函数通常可以用算法来设计。这个算法就是详细说明解决问题的方法。我们从 4 位数值比较器的设计推导描述这种方法的算法。

对于两个 4 位二进制数： $A=A_3A_2A_1A_0$ 和 $B=B_3B_2B_1B_0$ ，如果所有对应的位都相等，则这两个数相等。每组的相等关系可以用异或函数表示：

$$x_i = A_i B_i + A_i' B_i', \quad i = 0, 1, 2, 3$$

$$(A = B) = x_3 x_2 x_1 x_0$$

只有当两个数的所有对应位都相等时，该二进制变量 $(A=B)$ 才等于 1。

为了确定 A 和 B 的大小，从最高位开始检查对应的每组数的相对大小。如果两个数相等，就比较较低一位的一组数，一直比较到数不相等为止。

$$(A > B) = A_3 B_3' + x_3 A_2 B_2' + x_3 x_2 A_1 B_1' + x_3 x_2 x_1 A_0 B_0'$$

$$(A < B) = A_3' B_3 + x_3 A_2' B_2 + x_3 x_2 A_1' B_1 + x_3 x_2 x_1 A_0' B_0$$

符号 $(A > B)$ 和 $(A < B)$ 是二进制输出变量，分别在 $A > B$ 或 $A < B$ 时等于 1。 $(A = B)$ 、 $(A > B)$ 和 $(A < B)$ 三个输出变量逻辑门实现比较简单，包含了一定数量的重复元件，不相等的输出可以共用产生相等的输出的逻辑门。四位数值比较器如图 9.5 所示。

四个 x 输出由**异或非**电路产生，加到一个与门上以得到输出二进制变量 $(A = B)$ 。其他两个输出用变量 x 去产生前面列出的布尔函数组。这是一个多级实现并且有规则的方式。从这个例子可以清楚地了解获得多于四位的二进制数值比较器电路的方法。

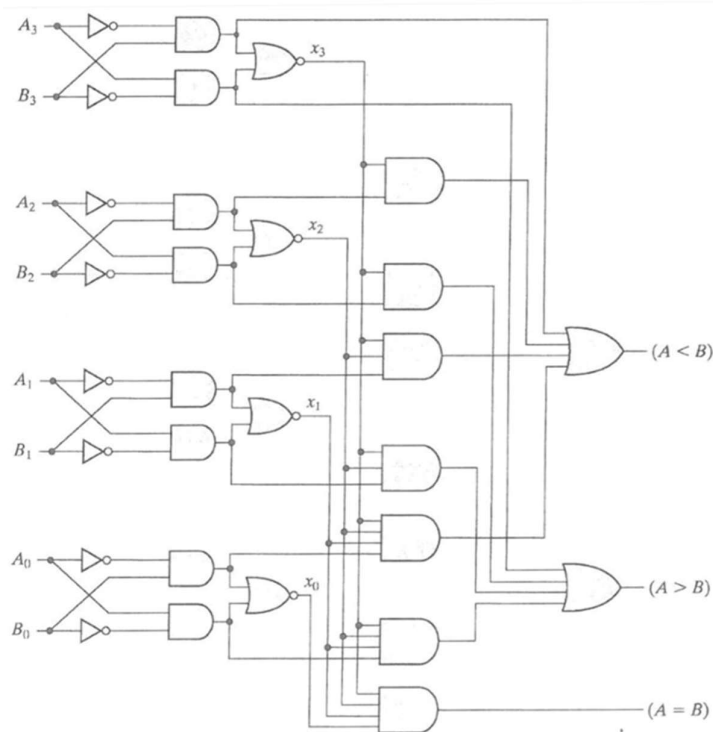


图 9.5 四位数值比较器逻辑电路图

第五节 译码器

数字系统中的离散信息量用二进制码来表示。一个 n 位二进制码可以表示 2^n 种不同的编码信息单元。译码器就是一个把来自 n 条输入线的二进制信息转化为 2^n 条输出线的组合电路。如果 n 位编码信息有没用的组合，译码器的输出可能少于 2^n 个。

上述译码器称为 n - m 线译码器，此处 $m < 2^n$ 。其目的是对 n 个输入变量产生 2^n (或小于) 个最小项，每种输入组合对应一个唯一的输出。译码器也可以用于其他关联的代码转换电路，例如 BCD-七段译码器。

以图 9.6 所示的 3-8 线译码器为例，3 个输入被译成 8 个输出，而每个输出代表 3 个输入变量的一个最小项，3 个非门提供反变量输入，8 个与门分别产生最小项中的一项。这种译码器的一个特殊应用就是二-十进制数的转换。输入变量代表一个二进制数，输出则代表了十进制系统中的 8 个数字。不管怎样，3-8 线译码器可以把任何 3 位码字译成八个输出，每个输出对应一个码字。

译码器可由与非门构成，以反相形式来产生译码器的最小项非常经济，也可以加上一个或多个使能端控制电路的工作。

一个由与非门构成的、带有一个使能输入的 2-4 线译码器如图 9.7 所示。

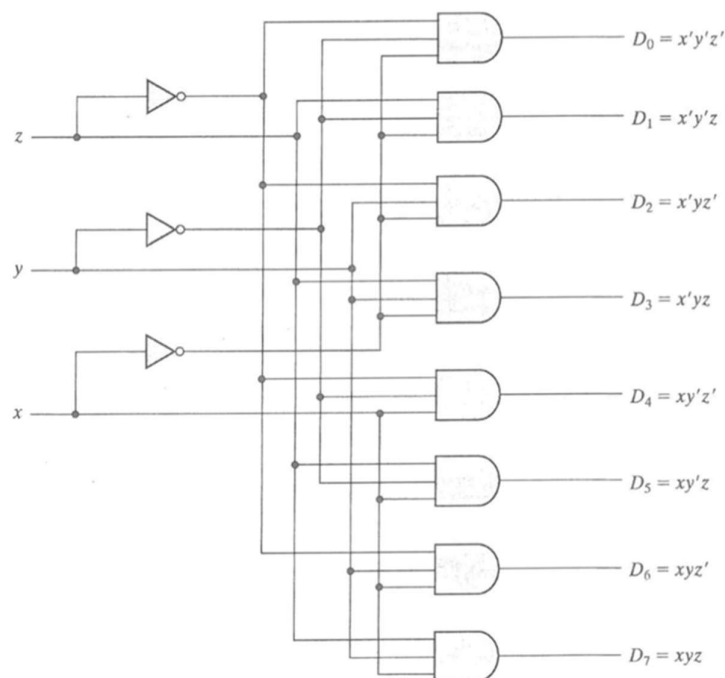


图 9.6 3-8 线译码器逻辑电路图

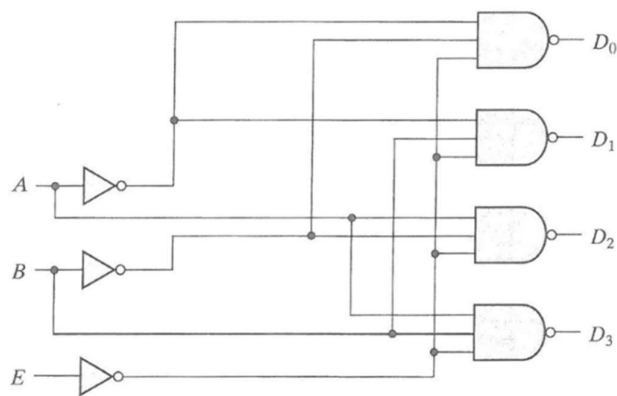


图 9.7 带使能输入的 2-4 线译码器

该电路的工作是以反相输出和低电平有效使能输入，当 E 等于 0 时译码器工作。可以看出，任何时候只有一个输出等于 0，其余所有的输出都为 1。值为 0 的输出代表了由输入 A 和 B 选择的最小项。

当 E 等于 1 时，不管其他两个输入值为何值，该电路被禁用。当此电路被禁用时，没有输出等于 0，因而没有最小项被选择。

总之，译码器可以采用反相输出也可以采用同相输出。使能输入端可用 1 或者 0 信号来激发。

有些译码器为了满足在给定逻辑条件下使电路开始工作，可以有两个或者更多的使

能输入。

带使能输入的译码器可用作数据选择器。数据选择器是一种从一条线路上接收信息，并可产生 2^n 条可能的输出线路的电路。一个特定输出的选择由 n 个选择线路的组合来控制。

当 E 作为数据输入线，而 A 、 B 作为选择输入时，图 9.7 所示的译码器可以当作 1-4 线数据选择器，唯一的输入变量 E 有到所有 4 个输出的路径，但是输入信息只由两条选择线 A 和 B 的二进制组合确定的其中一条路线输出，这一点可以由电路的真值表来验证。例如，如果选择线 $AB=10$ ，输出 D_2 就会和输入 E 的值一样，而所有其他的输出会保持在 1。因为译码器和数据选择器由相同的电路来实现，所以带使能端的译码器就称为译码器/数据选择器。

带使能端的译码器可以连在一起形成一个大型的译码器。例：两个带使能端的 3-8 线译码器连在一起构成了一个 4-16 线译码器。

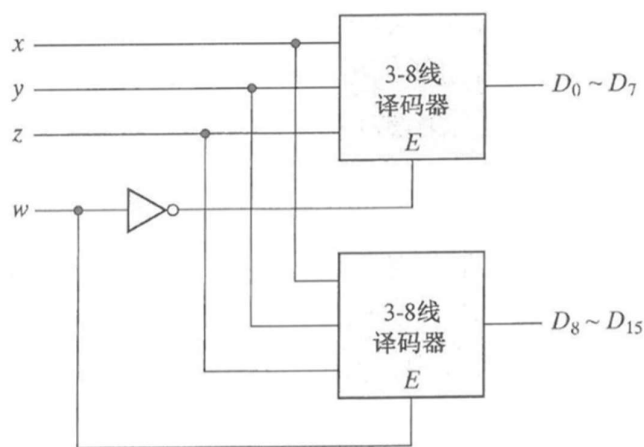


图 9.8 由两个 3-8 线译码器构成的 4-16 线译码器

当 $w=0$ 时，上面译码器工作，下面译码器不工作，下面译码器的所有输出都为 0，上面译码器产生从 0000 到 0111 的最小项。

当 $w=1$ 时，使能条件相反，下面译码器产生从 1000 到 1111 的最小项，而上面译码器的输出都是 0。

这个例子说明了使能输入在译码器以及其他组合逻辑单元中的作用。总的来说，使能输入对于连接两个或多个基本单元，以便将其扩展成为具有多个输入和输出的相同功能部件是非常有价值的。

用译码器实现组合逻辑

译码器提供 n 个输入变量的 2^n 个最小项，其输出由每一组输入唯一确定。任何布尔函数可以表示成最小项之和，就可以用译码器产生最小项，并且用外部或门来产生逻辑

和。用这种方法，任何有 n 输入、 m 个输出的组合电路就可以用一个 $n-2^n$ 译码器和 m 个或门来实现。

用译码器和或门实现组合电路的步骤要求：

先把电路的布尔函数用最小项之和的形式表示出来，然后选择译码器来产生输入变量的所有最小项，每个或门的输入从函数最小项译码器的输出中选择。

下面用一个全加器电路的实现来说明该步骤。

例 9.1 全加器电路的实现

采用最小项之和表示的组合电路的函数：

$$S(x, y, z) = \sum(1, 2, 4, 7),$$

$$C(x, y, z) = \sum(3, 5, 6, 7)$$

因为有 3 个输入和 8 个最小项，需要一个 3-8 线译码器，其实现如图 9.9 所示。

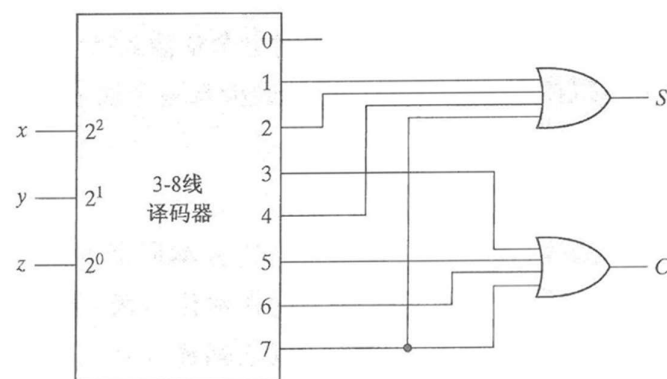


图 9.9 译码器实现组合逻辑的例子

译码器产生 x, y, z 的 8 个最小项，输出为 S 的或门产生最小项 1、2、4 和 7 的逻辑和，输出为 C 的或门产生最小项 3、5、6 和 7 的逻辑和。

有一长串最小项的函数需要一个多输入的或门。有 k 个最小项的函数可以表示为其 2^n-k 个最小项的反函数 F' 。

如果函数的最小项数目大于 $2^n/2$ ，那么 F' 可以用更少的最小项来表示。在这种情况下，用一个或非门来得到 F' 的最小项之和是很方便的。或非门的输出对这个逻辑和求反，产生正常的输出 F 。

如图 9.7 那样在译码器中使用与非门，则外部逻辑门必须是与非门而不是或门。这是因为两级与非门电路实现最小项函数的逻辑和，就相当于一个两级与或电路。

第六节 编码器

编码器是一种实现译码器反运算的数字电路，有 2^n 个输入线和 n 个输出线。例：8-3 线编码器，其真值表如表 9.4 所示，它有 8 个输入（每一组输入表示一个八进制数）和 3 个对应的二进制数输出。

假设在任何时候只有一个输入值等于 1。

编码器可以用或门来实现，其输入由真值表直接确定。当输入的八进制数是 1、3、5、7 时，输出 z 等于 1；对于八进制制数 2、3、6 或 7，输出 y 为 1；对于 4、5、6 或 7，输出 x 为 1。

表 9.4 8-3 线编码器真值表

输 入								输 出		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

表 9.4 所定义的编码器有一个局限，就是任何时候只有一个输入是有效的。如果两个输入同时有效，输入就会产生一个没有定义的组合。例如，如果 D_3 和 D_6 同时为 1，编码器输出将会是 111，三个输出都等于 1。这既不代表二进制数 3，也不代表二进制数 6。为了解决这个不确定因素，编码器电路必须建立输入优先机制，使得只有一个输出被编码。如果我们确定下标数高的输入有高的优先权，则当 D_3 和 D_6 同时为 1 时，输出将是 110， D_6 比 D_3 的优先级高。

8-3 线编码器中的另一个不确定性是，全 0 输出由全 0 输入产生；而这个输出与 D_0 等于 1 时的情况相同。这个矛盾可以用一个附加输出指示最少有一个输入等于 1 的方法来解决。

优先编码器

优先编码器是包含优先级函数的编码器电路。优先编码器中，如果有两个或多个输入同时有效，拥有最高优先级的输入将处于优先地位。

下表 9.5 是一个 4 输入优先级编码器的真值表。除了两个输出 x 和 y 以外，该电路还有第三个输出 V 。 V 是一个有效位指示符，当有一个以上的输入有效时，它就被设

定为 1。如果所有的输入无效， V 就等于 0。当 V 等于 0 时，另外的两个输出就被当成无关条件项而不用检查了。注意，输出栏中任何地方的 X 都代表无关条件，而输入栏中的 X 却对真值表的简化很有用。不用列出四个变量的所有 16 个最小项，真值表用一个 X 就代表了 1 或是 0。例如，X100 代表了两个最小项 0100 和 1100。

根据表 9.5，下标的数字越大，输入的优先级就越高。输入 D_3 具有最高的优先级，所以不管其他输入为何值，只要该输入为 1，输出 xy 就是 11（二进制数 3）。 D_2 具有第二优先权。如果 $D_3=0$ 而 $D_2=1$ ，不管其他两个低优先级的输入为何值，此时输出为 10。只有当更高优先级的输入为 0 时， D_1 对应的输出才会产生。下面以此类推。

表 9.5 4 输入优先级编码器的真值表

输 入				输 出		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

用于化简输出 x 和 y 的卡诺图如图 9.10 所示。两个输出函数的最小项是从表 9.5 推出的。尽管这个表只有 5 行，当一行中的每个 X 先后被 0 和 1 代替后，我们就得到了 16 个可能的输入组合。例如，表中第四行的 XX10 就代表了 4 个最小项 0010、0110、1010、1110。通过卡诺图化简，可以得到优先编码器的简化布尔表达式，输出 V 是所有输入变量的“或”。优先编码器的实现电路如图 9.11 所示，它对应于以下布尔函数：

$$\begin{aligned} x &= D_2 + D_3 \\ y &= D_1 D_2' + D_3 \\ V &= D_0 + D_1 + D_2 + D_3 \end{aligned}$$

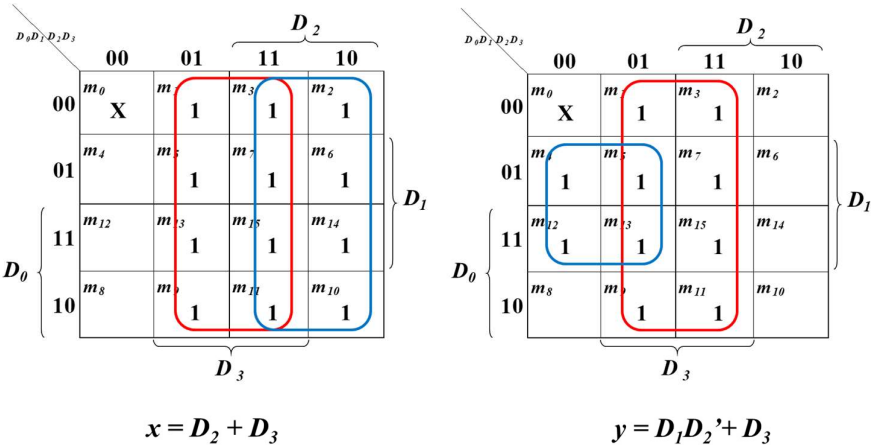


图 9.10 优先编码器卡诺图

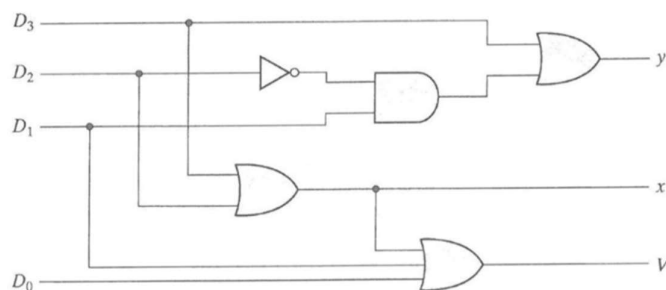


图 9.11 四输入优先编码器电路图

第七节 多路选择器

多路选择器（有时也称为数据选择器）是从多路输入线中选择其中的一路二进制信息到输出线上的一种组合电路。由一组选择线来控制，通常有 2^n 条输入线和 n 条选择线，选择线的组合决定哪条输入线被选中。

2 选 1 数据选择器是从两路二进制信源中选择一路连接到同一输出端，如图 9.12 所示。

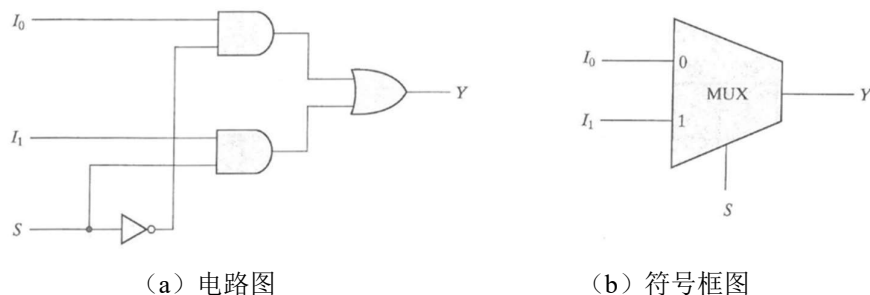


图 9.12 2 选 1 数据选择器

该电路有两条数据输入线、一条输出线，还有一条选择线 S 。当 $S=0$ 时，上面的与门工作， I_0 连接到输出。而当 $S=1$ 时，下面的与门工作， I_1 连接到输出。数据选择器就像个电子开关，用来选择两路信号中的一个信号，数据选择器的模块图有时用如图 9.12 (b) 所示的楔形符号表示，它直观地显示了某个被选中的多路数据源如何被传送至单个终端。数据选择器用符号 MUX 来表示。

图 9.13 (a) 所示的传输门电路常常作为 CMOS 工艺实现的多路选择器。这一电路用 4 个 MOSFET 实现路径选择逻辑（每个传输门有两个晶体管）。如果包括一对用于选择信号的缓冲反相器，那么总的晶体管数目增加到 8 个。该电路的主要问题是传输门具有寄生电阻和电容，会使响应减慢。图 9.13 (b) 为只用 NMOS 传输管的相同电路形式。包括选择信号驱动器在内，晶体管数目减少到 6 个。但在输出端增加了一个反相驱动器以补偿 NMOS 只能通过电压范围 $0V$ 至 $V_{\max}$ 的问题： $V_{\max} = V_{DD} - V_{Tn}$ 。这一反相器帮助把输出恢复至全摆幅的电压范围 $[0, V_{DD}]$ 。尽管这时晶体管数目与用传输门电路时相同，

但版图的布线将比较容易。

较大的多路选择器可以用基本门或级联 2 选 1 数据选择器来设计。4 选 1 数据选择器如图 9.14 所示。 I_0 到 I_3 的 4 个输入中每一个都接到一个与门的输入端。选择线 S_1 和 S_0 用来选择某一特定的与门。各与门的输出加到一个提供单线输出的或门。函数表中列出的是根据选择线的不同二进制组合传送至输出的数据输入。为了描述该电路的操作，考虑 $S_1S_0=10$ 的情况。从图 9.14 可知， I_2 所连的与门有两个输入等于 1，第三个输入为 I_2 ，其他的三个与门至少有一个输入等于 0，其输出就等于 0，此时或门的输出就等于 I_2 的值，提供了一条从选中的输入到输出的路径。

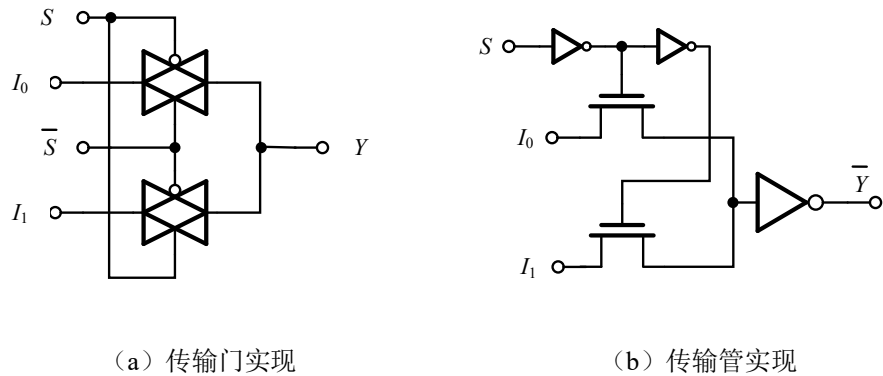


图 9.13 用传输门和传输管实现 2 选 1 多路选择器

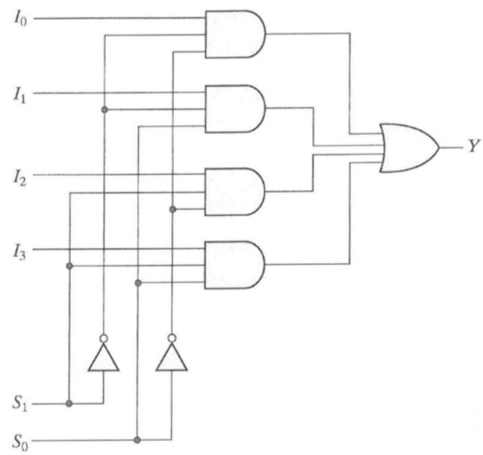


图 9.14 4 选 1 数据选择器电路图

数据选择器中的与门和非门实际上类似于译码器电路，它们对所选的输入线进行译码。

总之，一个 2^n 选 1 的数据选择器由一个 $n-2^n$ 译码器加上 2^n 个输入线构成，每个输入线接到一个与门上，与门的输出再接到一个或门上。数据选择器的规模由 2^n 个数据输入线和一个输出线决定。 n 个选择线就意味着有 2^n 个数据线。如译码器那样，数据选择器可以有一个使能输入来控制该部件的功能。当使能输入无效时，输出被禁止；而当使

能输入有效时，电路的功能就是一个正常的数据选择器。

数据选择器电路可以通过选择输入的组合来提供多位选择逻辑。例如，一个四位 2 选 1 数据选择器如图 9.15 所示。

该电路有四个数据选择器，每个可以选择两个输入线中的一个。输出 Y_0 可以选择 A_0 或 B_0 。类似地，输出 Y_1 的值可能是 A_1 或 B_1 ，依此类推。

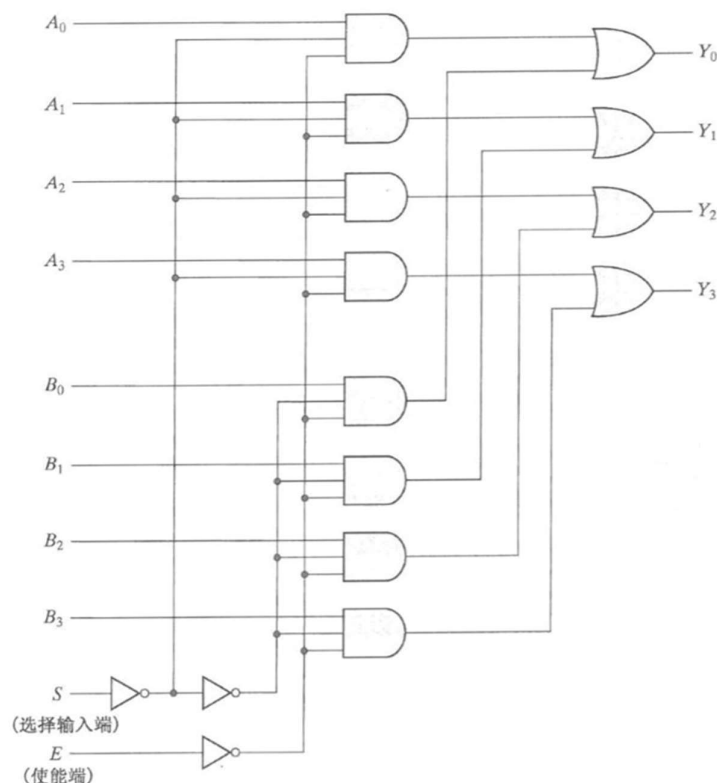


图 9.15 四位 2 选 1 数据选择器电路

输入选择线 S 选择 4 个数据选择器中的一组输入。正常工作时，使能输入 E 必须处于有效状态。尽管该电路包含了 4 个二选一数据选择器，但我们更喜欢把它视为一个从两组 4 位数据线中选择一组数据的电路。当 $E=0$ 时，该单元工作，若 $S=0$ ，则 A 数据的 4 个输入有一条通道通向 4 个输出。另一方面，若 $S=1$ ，则 B 数据的 4 个输入被传送到输出。而当 $E=1$ 时，不管 S 为何值，输出都为 0。

用数据选择器实现布尔函数

前面介绍过译码器外加或门可以用来实现布尔函数。由数据选择器逻辑图可以看出，数据选择器实质上就是一个带或门的译码器。在数据选择器中，函数的最小项由与选择输入相连的电路产生，单个最小项可以由输入数据来选择。

这里提供一种方法，可以用一个有 n 个选择输入和 2^n 个数据输入的数据选择器来实现布尔函数，其中每个数据输入对应一个最小项。

另一种更有效的方法是，用一个有 $n-1$ 个选择输入的数据选择器来实现 n 个变量的布尔函数。函数的前 $n-1$ 个变量连到数据选择器的选择输入端，剩下的一个变量用作数据输入。

如果是单个变量 z ，那么数据选择器的每个数据输入将会是 z 、 z' 、1 或 0。例如下面的三变量布尔函数：

$$F(x, y, z) = \sum(1, 2, 6, 7)$$

该三变量函数可以用图 9.16 所示的 4 选 1 数据选择器来实现。

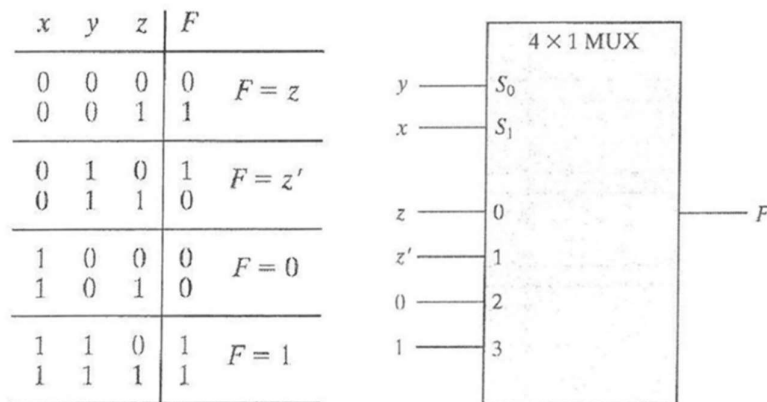


图 9.16 用数据选择器实现三变量布尔函数

两个变量 x 和 y 按顺序提供给选择线； x 连接到 S_1 输入端， y 接到 S_0 输入端。数据输入线的值由函数真值表决定。当 $xy=00$ 时，输出 F 等于 z ，这是因为当 $z=0$ 时， $F=0$ ，而当 $z=1$ 时， $F=1$ 。这就要求 z 变量被赋给数据输入 0。该数据选择器的操作是这样的：当 $xy=00$ 时，数据输入 0 有一条路径通到输出并且使 F 等于 z 。相同的方式下，可以根据当 $xy=01$ 、10 和 11 时， F 的值各不相同来确定所需的输入数据线 1、2 和 3。这个特殊的例子描述了数据输入中可以获得的所有四种可能。

用 $n-1$ 个选择输入和 2^{n-1} 个数据输入的数据选择器实现任意 n 个变量的布尔函数，一般步骤可参照前面的例子。

- 首先列出布尔函数的真值表，真值表的前 $n-1$ 个变量被用作数据选择器的选择输入；
- 对于输入变量的每个组合，把输出当作最后一个变量的函数，这个函数可以是 0、1、原变量、反变量；
- 将这些值以正确的顺序加到数据输入端 D 。

例如下面的布尔函数：

$$F(A, B, C, D) = \sum(1, 3, 4, 11, 12, 13, 14, 15)$$

该函数用一个 3 选择输入的数据选择器来实现，如图 9.17 所示。

注意，第一个变量 A 必须接选择输入 S_2 ，这样 A 、 B 和 C 就分别对应选择输入 S_2 、 S_1 和 S_0 。

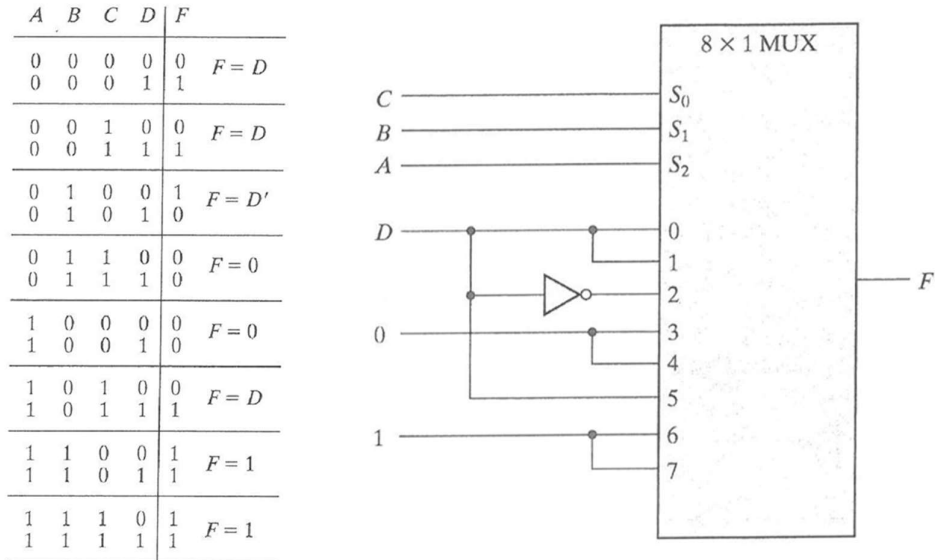


图 9.17 用数据选择器实现 4 输入布尔函数

数据输入线所接的输入由真值表列出的数字决定。相应的数据线由 ABC 的二进制组合决定。例如，当 $ABC=101$ 时，表中显示 $F=D$ 。这样输入变量 D 就提供给数据输入线 D_5 。

二进制常数 0 和 1 对应两个固定的信号值，当用于集成电路时，逻辑 0 对应于信号地，而逻辑 1 相当于电源信号（比如 3V），具体取决于电路的工艺技术。

三态门

数据选择器可以用三态门来构造。三态门是一种呈现三种状态的数字电路，其中的两种状态相当于传统的逻辑门的逻辑 1 和 0，第三种状态是高阻态。

高阻态呈现为开路，也就是输出表现为无连接，该电路此时没有逻辑意义。

三态门可以用来实现任何一个普通逻辑门，如与门和与非门。不过，最常用的还是用作缓冲门。

三态缓冲器的图形符号如图 9.18 所示，它通过在门的图标底部加上一条输入控制线来和普通的缓冲器相区别。

该缓冲器有一个数据输入 A 、一个输出 Y 和一个决定输出状态的控制输入 C 。当控

制输入 C 等于 1 时，输出使能，该逻辑门作为普通的缓冲器，输出 Y 就等于正常的输入 A 。当控制输入 C 为 0 时，输出无效，不管正常输入为何值，该逻辑门输出处于高阻态。

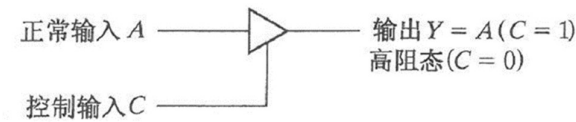


图 9.18 三态缓冲器图形符号

三态门的高阻态提供了一种其他门没有的特性。因为这个特性，可以将大量的三态门输出用线连在一起形成一个共用线。

用三态门组成的数据选择器结构如图 9.19 所示。图 9.19 (a) 部分显示了用两个三态门和一个反相器构成的 2 选 1 数据选择器的结构。

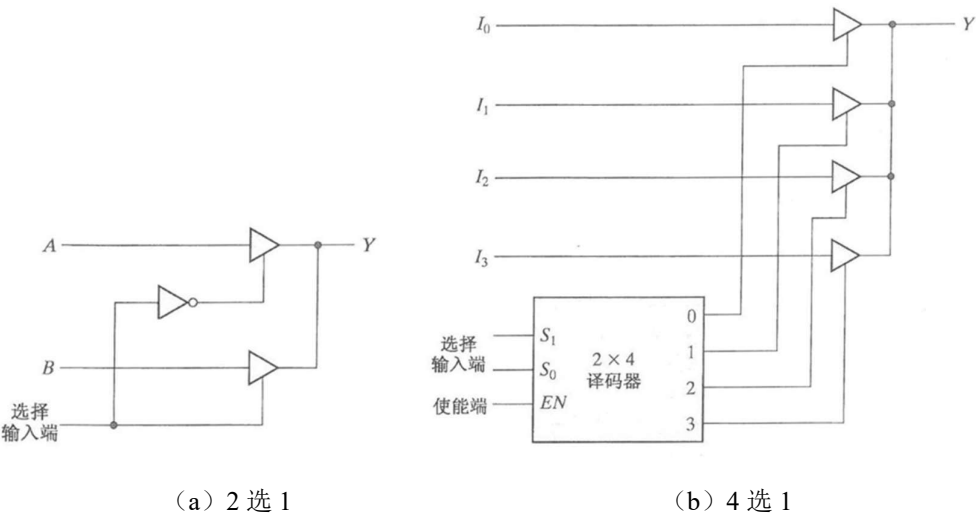


图 9.19 用三态门构成的数据选择器

两个输出被连接在一起以形成单条输出线（通常非三态输出逻辑门不能做这种连接）。当选择输入为 0 时，上面的缓冲器由输入控制使能，而下面的缓冲器无效，输出 Y 等于输入 A ；当选择输入为 1 时，下面的那个缓冲器使能，输出 Y 等于 B 。

4 选 1 数据选择器结构如图 9.19 (b) 所示。四个三态门的输出连在一起形成单一输出线。缓冲器的控制输入决定四个普通输入 I_0 到 I_3 其中之一将与输出线相连。任一时间，起作用的缓冲器数不会超过一个。已连接的缓冲器必须得到控制，以便只有一个三态缓冲器通向输出，而所有其他缓冲器均保持在高阻态。为确保在任何时候只有一个控制输入有效的方法是用译码器。当译码器的使能输入为 0 时，它的四个输出全是 0，因为所有的四个缓冲器都被禁止，所以输出总线呈现高阻态。当使能输入有效时，译码器的选择输入值将确定其中一个三态缓冲器有效。仔细研究将会发现，该电路以另外一种方式构造了一个 4 选 1 数据选择器。

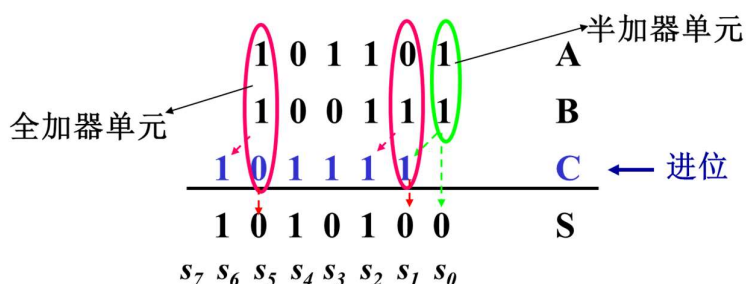
第八节 加法器设计

9.8.1 二进制加法器

数字计算机能实现多种不同的信息处理任务。在遇到的函数中有多种多样的数学运算，绝大多数基本的数学运算是两个二进制数的加法。这个简单加法有四种可能的基本算式： $0+0=0$ 、 $0+1=1$ 、 $1+0=1$ 和 $1+1=10$ 。前三种算式产生的和是一位数，但是当加数和被加数都为 1 时，二进制数的和就由两位数组成，运算结果的最高有效位被称为进位。若加数和被加数包含了多个有效数字，则两个数相加后的进位就被送至相邻的高位。

$$\begin{array}{r} \text{被加数:} \quad 101101 \\ \text{加数:} \quad + \quad 100111 \\ \hline \text{和:} \quad 1010100 \end{array}$$

二进制加减器是实现二进制数加法和减法运算的组合电路。其中实现两个二进制比特相加的组合电路称作半加器（Half-Adder, HA）；实现三位二进制数（两个有效数和一个进位输入）加法的组合电路是全加器（Full-Adder, FA）。这样命名是因为两个半加器组合在一起可以实现一个全加器的功能。



下面，首先进行半加器的设计，然后再将其升级到全加器。将 n 个全加器进行级联，可实现 n 位二进制加法器。

半加器（HA）

实现两位二进制数加法，需要两个二进制输入和两个二进制输出。两个输入记为 x 和 y ，两个输出记为 S （和）和 C （进位）。半加器的真值表列在表 9.6 中。只有当两个输入全为 1 时，输出 C 才是 1。输出 S 代表和的最低有效位。两个最简输出布尔函数可由真值表直接得到：

$$\begin{aligned} S &= x'y + xy' \\ C &= xy \end{aligned}$$

半加器的积之和式逻辑实现如图 9.20 (a) 所示。它也可以用一个异或门和一个与门

来实现，如图 9.20（b）所示。这种实现方式显示了两个半加器可以构成一个全加器。

表 9.6 半加器真值表

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

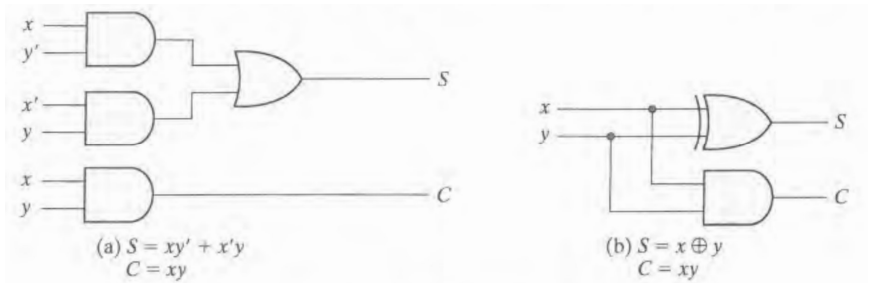


图 9.20 半加器的实现电路

全加器（FA）

全加器实现 3 位二进制数算术和，它有 3 个输入和两个输出。两个输入变量 x 和 y 分别代表两个待加的有效位，第三个输入变量 z 代表来自低有效位上的进位；输出必须有两个，因为三个二进制数算术和的范围是 0 到 3，而二进制数 2 或 3 的表示则需要两位。两个输出变量中符号 S 代表和，符号 C 代表进位。 S 给出了和的最低有效位值， C 给出了输出进位值，该进位由当前位与前一位的进位求和产生。全加器的真值表列在表 9.7 中。输入变量栏里的八行给出了输入的三个变量所有可能的组合，输出变量由输入位的算术和决定。当输入全为 0 时，输出也是 0。当仅有一个输入等于 1 或者三个输入都等于 1 时，输出 S 等于 1。如果两个或者三个输入等于 1，输出进位 C 等于 1。

表 9.7 全加器的真值表

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

全加器两个输出布尔函数经化简后可得到：

$$\begin{aligned}
 S &= x'y'z + x'yz' + xy'z' + xyz \\
 C &= xy + yz + xz
 \end{aligned}
 \tag{9.1}$$

全加器逻辑图积之和的实现结果如图 9.21 所示。

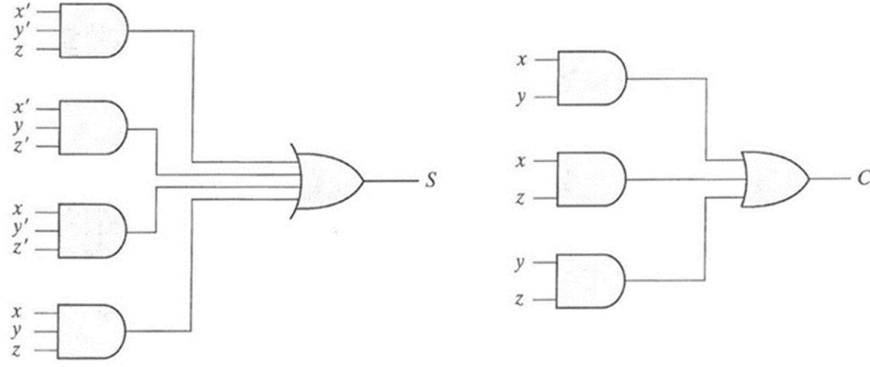


图 9.21 积之和式全加器的实现

全加器电路也可用两个半加器和一个或门实现，如图 9.22 所示。第二个半加器的输出 S 由 z 和第一个半加器的输出相异或而得：

$$\begin{aligned}
 S &= z \oplus (x \oplus y) = z'(x'y + xy') + z(x'y' + xy) \\
 &= x'y'z + x'yz' + xy'z' + xyz \\
 C &= z(x'y + xy') + xy = x'yz + xy'z + xy
 \end{aligned}
 \tag{9.2}$$

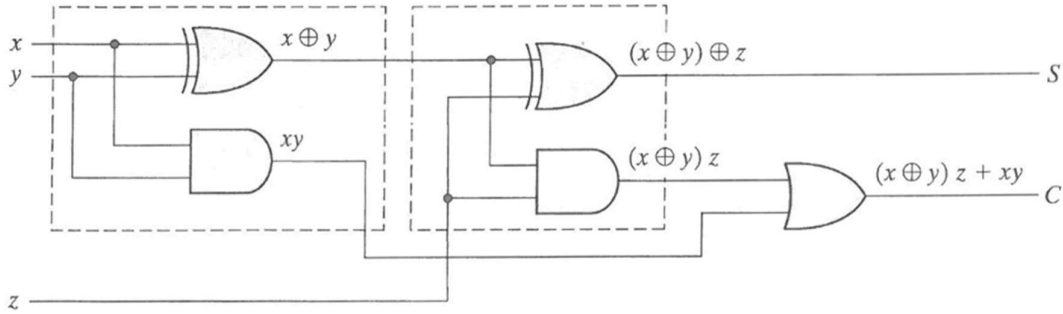


图 9.22 用两个半加器和一个或门实现的全加器

二进制加法器

二进制加法器是用来产生两个二进制数算术和的数字电路。它可以通过全加器的级联来实现，即每个全加器的进位输出和下一个全加器的一个输入相连，形成链状结构。 n 位二进制数加法需要 n 个全加器级联，或一个半加器和 $n-1$ 个全加器级联实现。在前者情况下，最低有效位上的输入进位需要置 0。图 9.23 所示为 4 个全加器级联而成的 4 位逐位进位加法器或行波进位加法器 (*ripple-carry adder*)，被加数 A 和加数 B 由带下标的字母从右到左指定，下标为 0 就代表最低位。全加器之间的进位连接成链状。加法器的输入进位为 C_0 ，它依次穿过四个全加器到达输出进位 C_4 。输出 S 产生需要的二进制

和。每个全加器的输出进位连接到高位全加器的输入进位上，这样 n 个全加器就能构成一个 n 位加法器。

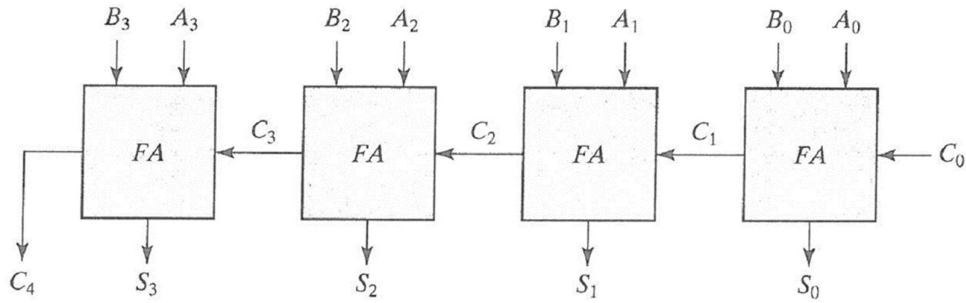


图 9.23 四位加法器

并行的两个二进制数加法是各位加数与被加数同时计算。对于任何组合电路，信号必须经过逻辑门的传输才能在输出端得到正确的运算结果。总的传输时间等于一个典型门的延时乘以电路中逻辑门的级数。加法器中最长的传播延时是进位通过各全加器所用的时间。既然输出结果的每一位都与输入进位有关，那么只有当输入进位已经传输到该级，全加器任何一级的 S_i 值才是稳定的。例如图 9.23 中的输出 S_3 ，输入信号一加到加法器上，输入 A_3 和 B_3 就有效。然而直到前一级进位 C_2 送到，输入进位 C_3 才能稳定在最终的值。以此类推， C_2 要等 C_1 ，一直等到 C_0 。这样，只有在进位传输穿过各级电路后，最终的输出 S_3 与进位 C_4 才能稳定在正确的值。

通过该电路的延时取决于传播必须通过的逻辑级的数目并且与所加的输入信号有关。对于某些输入信号完全不会发生行波进位效应，而对于另一些信号，进位必须从最低有效位 (*least significant bit*, LSB) 一直波动到最高有效位 (*most significant bit*, MSB)。这样一个结构（也称为关键路径，*critical path*）的传播延时定义为对所有可能的输入图形在最坏情形下的延时。

在逐位进位加法器中，最坏情形的延时发生在当最低有效位上产生的进位一直全程传播到最高有效位时。这一进位最终在最后一级上被吸收以产生和。因此，延时正比于输入字的位数 N 并近似为：

$$t_{adder} \approx (N-1)t_{carry} + t_{sum} \quad (9.3)$$

式中， t_{carry} 和 t_{sum} 分别等于全加器中从输入进位 (C_i) 至输出进位 (C_o) 及和 (S) 的传播延时。

进位传输经过门的级数可以从全加器电路中得到。为方便起见，我们把电路重绘于图 9.24 中。输入和输出变量用下标 i 代表加法器的 i 级。 P_i 和 G_i 信号在传输过程中要保持稳定，这两个信号对所有的全加器来说仅仅由输入的二进制加数与被加数来确定。从输入进位 C_i 到输出进位 C_{i+1} 的信号传输经过了一个与门和一个或门，构成了两级门电路。如果加法器里有四个全加器，则输出进位 C_4 必须穿过 C_0 和 C_4 之间的 $2 \times 4 = 8$ 级门

电路。对于 n 位加法器，从输入进位到输出进位需要穿过 $2n$ 级门电路。

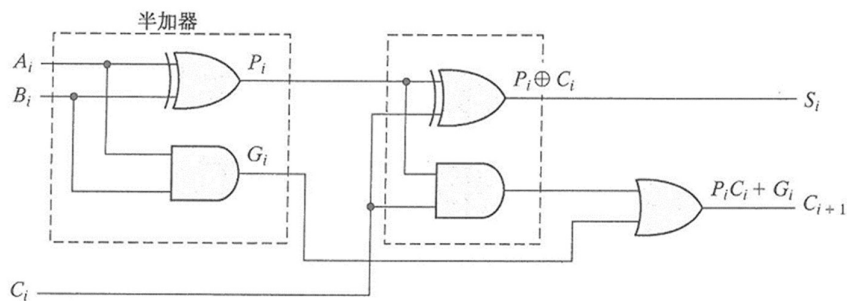


图 9.24 带有 P 和 G 的全加器

考虑图 9.24 所示的全加器电路，如果我们定义如下二进制变量

$$G_i = A_i B_i$$

$$D_i = \bar{A}_i \bar{B}_i$$

$$P_i = A_i \oplus B_i \quad (9.4)$$

输出的和与进位可表示为：

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i \quad (9.5)$$

G_i 称为进位产生，不管输入进位 C_i 为何值，当 A_i 和 B_i 都为 1 时，它产生一个进位。 P_i 称为进位传播，因为它与从 C_i 到 C_{i+1} 的进位传播有关。 D_i 称为进位取消，不管输入进位 C_i 为何值，当 A_i 和 B_i 都为 0 时，将取消一个进位。

例 9.2 逐位进位加法器的传播延时

推导 A_k 和 B_k ($k=0, \dots, N-1$) 的值以得到逐位进位加法器最坏情形下的延时。最坏情形是在 LSB（最低有效位）上产生了一个进位，由于第一个全加器的输入进位 C_0 总是 0，这意味着 A_0 和 B_0 必须同时等于 1。其他所有各级必须处于传播模式。因此， A_i 或 B_i 必为高电平。最后，我们可以实际度量一下在 MSB（最高有效位）的和位上发生翻转的延时。假设 S_{N-1} 的初值为 0，我们必须安排一个 $0 \rightarrow 1$ 的翻转。这可以通过使 A_{N-1} 和 B_{N-1} 同时为 0（或同时为 1）来实现，于是在输入进位为 1 时就产生一个高电位的和位。

例如，以下 A 和 B 的值将导致一个 8 位加法最坏情形下的延时：

$$A: 00000001; B: 01111111$$

为了得到一个延时最长的翻转，可以使所有的输入在 A_0 由 $0 \rightarrow 1$ 翻转时保持为常数。

最左边的位代表这个二进制表达式中的最高有效位。注意，这只是许多最坏情形中的一个。这一情形使最终的和经历 $0 \rightarrow 1$ 的过渡延时。

进位传输时间对两个数相加的速度是限制因素。尽管加法器或所有组合电路在其输出端总会有一些值，但是如果不能给予足够的时间让信号传输通过输入和输出之间的门电路，其输出就不会正确。既然所有其他的数学运算都可以用连续加法来实现，因此加法过程中的耗时就很关键。减少进位时延的一个有效方法就是使用速度快、时延小的逻辑门电路。然而，电路的物理性能总是有限的。另一个解决办法是提高电路的复杂度以减少进位时延。目前，已有多种技术可以降低并行加法器的进位传播时延。

从公式 (9.3) 中可以得出两个重要结论：

- 逐位进位加法器的传播延时与 N 成线性关系。在设计当前和未来计算机所希望的宽数据通路 ($N = 16, \dots, 128$) 加法器时这一点变得日益重要。

- 在设计一个快速逐位进位加法器的全加器单元时，优化 t_{carry} 比优化 t_{sum} 重要得多，因为后者对 t_{adder} 总延时值的影响较小。

在深入讨论全加器单元的电路设计之前，值得提及的是全加器的另一个逻辑性质：

把一个全加器的所有输入反相，它的所有输出也反相。

这一性质也称为加法器的反相特性，可用下列两个公式来描述：

$$\begin{aligned}\bar{S}(A, B, C_i) &= S(\bar{A}, \bar{B}, \bar{C}_i) \\ \bar{C}_o(A, B, C_i) &= C_o(\bar{A}, \bar{B}, \bar{C}_i)\end{aligned}\quad (9.6)$$

这在优化逐位进位加法器的速度时非常有用。它说明图 9.25 左右两个电路（在逻辑关系上）完全等效。

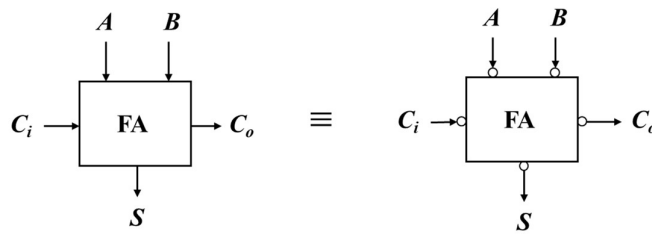


图 9.25 全加器的反相特性（图中的小圆圈代表反相）

9.8.2 全加器：电路设计考虑

静态加法器电路

实现全加器电路的一种方法是把逻辑方程 (9.1) 直接转变为互补 CMOS 电路。进行某些逻辑变换可以帮助减少晶体管的数目。例如，只要不减慢进位产生的速度（如前所述进位产生是最关键的部分）则在“和”与“进位产生”的子电路之间共享某些逻辑非常有利。以下是这样一个重新组织方程组的例子：

$$C_o = AB + BC_i + AC_i \quad (9.7)$$

和

$$S = ABC_i + \bar{C}_o (A + B + C_i)$$

很容易验证它与原方程组等价。相应的采用互补静态 CMOS 的加法器设计见图 9.26, 它需要 28 个晶体管。除了消耗较大的面积而外, 这一电路的速度也较慢。因为:

- 在进位产生与和产生电路中堆叠着许多 PMOS 管。
- C_o 信号的本征负载电容很大, 包括两个扩散电容、6 个栅电容, 再加上连线电容。
- 在进位产生电路中信号传播通过两个反相级。正如前面提到的, 最小化进位路径的延时是高速加法器电路设计者的首要目标。如果进位链输出上的负载 (扇出) 很小, 那么有两个逻辑级就显得太多, 因而引起额外的延时。
- 和的产生要求一个额外的逻辑级, 但这并不那么重要。因为这一项在逐位进位加法器的传播延时公式 (9.3) 中只出现一次。

这一电路尽管慢, 但却体现了某些聪明的设计技巧。注意, 在进位产生电路第一个门的设计中 C_i 信号放在串联数较少的 PMOS 管支路上从而使它的逻辑努力降低为 2。同时, 连接到 C_i 的 NMOS 和 PMOS 管尽可能地放在靠近这个门的输出端。这直接应用了 6.4.2 节中讨论过的电路优化技术——关键路径上的晶体管应当尽可能放在接近门的输出端。例如在加法器的第 k 级, 信号 A_k 和 B_k 在 $C_{i,k} (=C_{o,k-1})$ 到达之前就已经存在并已稳定了很长时间。因此, 在晶体管链中内部节点的电容事先已被预充电或放电。当 $C_{i,k}$ 到达时, 只有节点 X 的电容必须充 (放) 电。如果把晶体管放在靠近 V_{DD} 和 GND 处, 则不仅需要充 (放) 电节点 X 的电容, 而且也要充 (放) 电内部节点的电容。

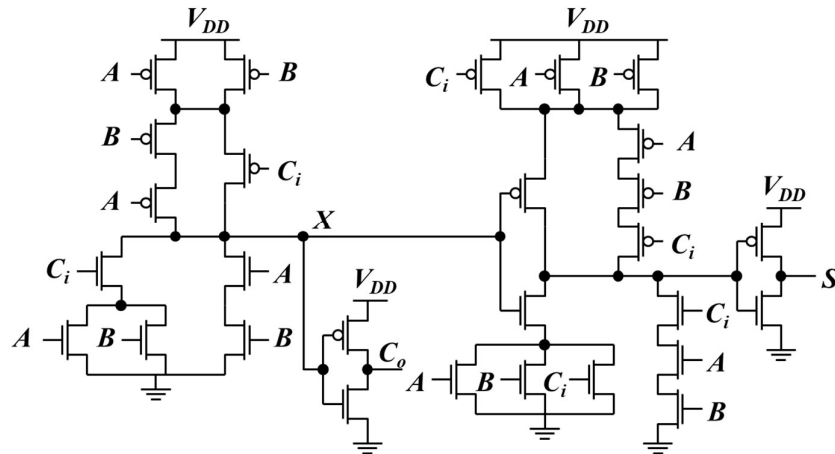


图 9.26 互补静态 CMOS 实现的全加器

现在, 这一电路的速度可以利用前面讨论过的加法器的某些性质逐步加以改善。首先, 利用加法器的反相特性, 即把一个全加器单元的所有输入反相则它的所有输出也反相, 这可以减少进位路径中反相级的数目。如图 9.27 所示, 这一特性允许我们在一个进

位链中消除反相器。

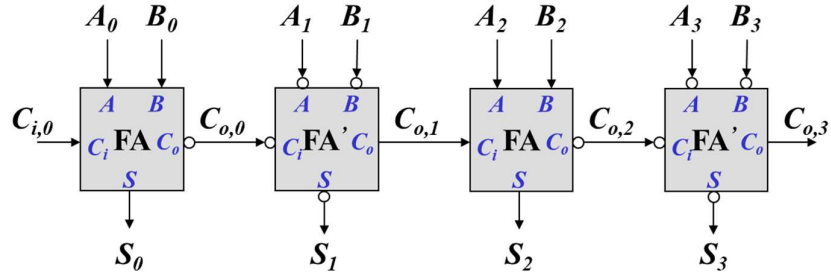


图 9.27 在进位路径中取消反相器（FA' 表示进位路径中没有反相器的全加器）

镜像加法器设计

图 9.28 是一个经过改进的加法器电路，也称为镜像加法器（*mirror adder*）。它的工作基础是公式 (9.5)，但它的进位产生电路值得分析。首先，如前面所建议的，取消了进位反相门。其次，门的 PDN 和 PUN 网络不再是对偶的，而是巧妙地实现了进位传播/产生/取消功能——当 D 或者 G 为高时， $\bar{C}_o$ 分别被置为 V_{DD} 或 GND 。满足进位传播条件时（即 $P = A+B$ 为 1）输入进位（以反相的形式）传播到 $\bar{C}_o$ 。这一结构使面积和延时都有相当程度的减少。以下一些事实值得考虑：

- 该全加器单元仅需要 24 个晶体管。
- NMOS 和 PMOS 链完全对称，由于求和与进位功能的自对偶性（*self-duality*）它同样能产生正确的功能。但这一结果使在进位产生电路中最多只有两个晶体管串联。
- 连接 C_i 的晶体管放在最接近门的输出端处。
- 只有在进位电路中的晶体管才需要优化尺寸以改善速度。在求和电路中的所有晶体管都可以采用最小尺寸。当设计该单元的版图时，最关键的问题是使节点 $\bar{C}_o$ 处的电容最小。共享扩散区可以减少堆叠（晶体管）的节点电容。
- 在图 9.26 的加法器单元中，可以独立确定反相器的尺寸来驱动下一级加法器的 C_i 输入。如果图 9.28 中的进位电路尺寸对称，则其每一个输入的逻辑努力为 2。这意味着优化尺寸以达到最小延时的最优扇出数应当为 $(4/2) = 2$ 。然而，这一级的输出驱动两个内部的栅电容和 6 个所连下一级加法器单元的栅电容。保持每一级晶体管尺寸相同的一个巧妙办法是把进位级的尺寸增加至大约为求和级尺寸的 3 至 4 倍。这仍保持优化扇出为 2。所得到的晶体管的尺寸标在图 9.28 中，这里假设 PMOS/NMOS 的比率为 2。

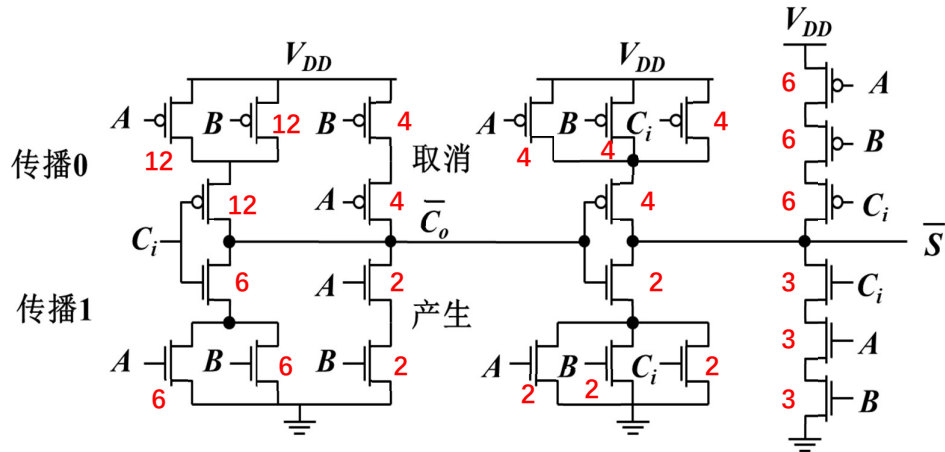


图 9.28 镜像加法器电路图

传输门型加法器

一个全加器也可以采用多路开关和 XOR 来设计。虽然这在互补 CMOS 实现中并不可行，但当多路开关和 XOR 采用传输门实现时它就变得很有吸引力。图 9.29 即为基于这一方法实现的一个全加器，它使用了 24 个晶体管。它来源于公式 (9.5) 的进位传播产生模型。进位传播信号即输入 A 和 B 的 XOR，用来选择输入进位值的正信号或反信号作为新的和输出。根据进位传播信号值的不同，输出进位或者为输入进位，或者为输入 A 和 B 的其中之一。这种加法器的一个有意义的优点是它的和与进位输出具有近似的延时。

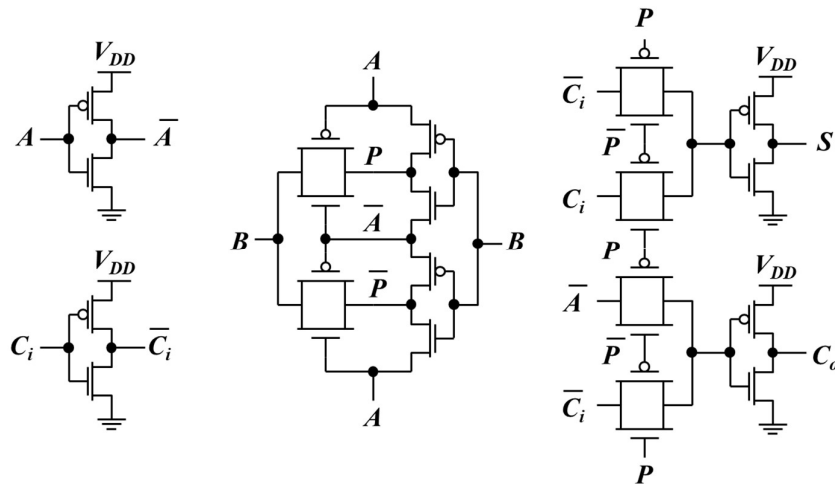


图 9.29 求和与进位延时相近的传输门型全加器

曼彻斯特 (Manchester) 进位链加法器

图 9.29 中的进位传播电路可以通过增加进位产生和进位消除信号来简化, 如图 9.30 (a) 所示。进位传播路径不充电, 如果进位传播信号 ($A_i \oplus B_i$) 为真, 则 C_i 被传送到输出 C_o 。如果传播条件不满足, 则输出或者由信号 D_i 下拉或者由 $\bar{C}_i$ 上拉。动态实现[见图 9.30 (b)]甚至可以使电路更加简单。由于动态电路中的晶体管是单方向工作的, 所以可以只用 NMOS 传输管来代替传输门。预充电输出使电路不再需要进位取消信号 (对于进位链传播的是进位信号的反信号时的情形)。

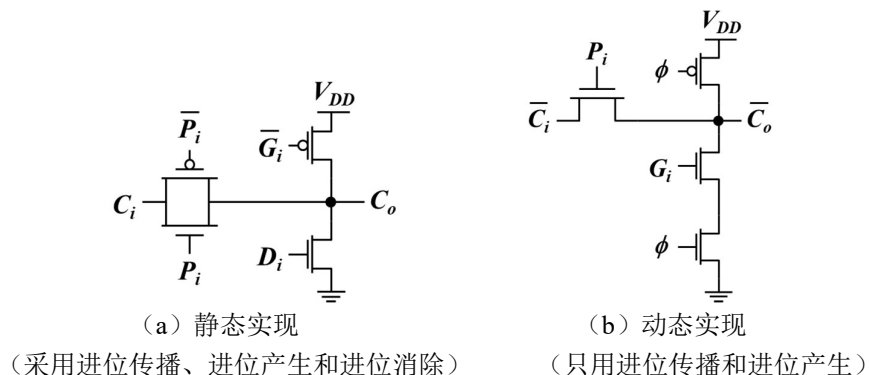


图 9.30 曼彻斯特进位门

曼彻斯特进位链加法器用串联的传输管来实现进位链。图 9.31 是以图 9.30 介绍的动态电路为基础的一个例子。在预充电阶段 ($\phi = 0$), 传输管进位链中的所有中间节点都被预充电至 V_{DD} 。在求值阶段, 当有输入进位且传播信号 P_k 为高电平时, 或者当第 k 级的进位产生信号 (G_k) 为高电平时, 节点 $\bar{C}_k$ 放电。

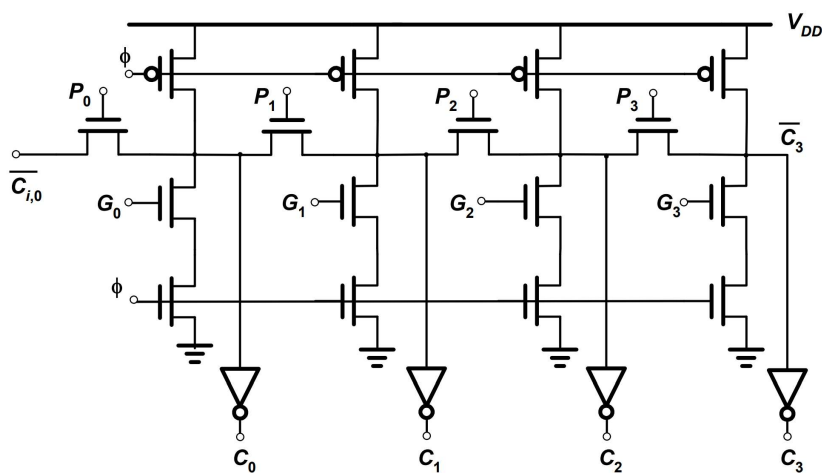


图 9.31 以动态逻辑实现的曼彻斯特进位链加法器 (四位部分)

将图 9.31 中加法器进位链在最坏情形下的延时模拟成图 9.32 中的线性 RC 网络。如在第 5 章中所推导的, 当所有的 $C_i = C$ 且 $R_i = R$ 时, 这样一个网络的传播延时等于:

$$t_p = 0.69 \sum_{i=1}^N C_i \left(\sum_{j=1}^i R_j \right) = 0.69 \frac{N(N+1)}{2} RC \quad (9.8)$$

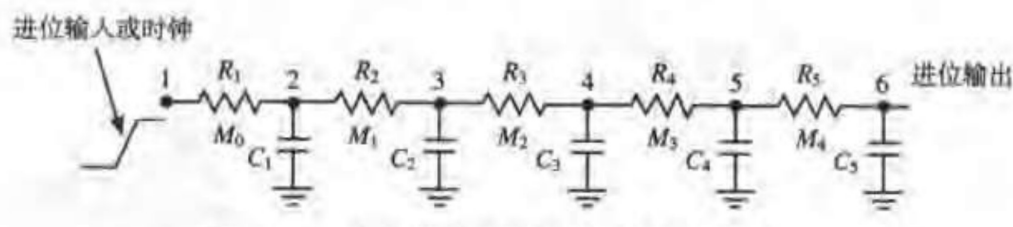


图 9.32 决定进位链传播延时的等效网络

例 9.3 确定曼彻斯特进位链的尺寸

进位链上每一节点的电容等于四个扩散电容、一个反相器输入电容以及与这一单元尺寸成正比的导线电容。反相器和 PMOS 预充电晶体管可以保持在单位尺寸，加上导线电容，固定电容可以估计为 15fF（就我们的工艺而言）。如果一个宽度为 W_0 的单位尺寸晶体管的电阻为 $10\text{K}\Omega$ ，扩散电容为 2fF，则宽度为 W 的晶体管链的 RC 时间常数为：

$$RC = (6\text{fF} \cdot (W/W_0) + 15\text{fF}) \cdot 10\text{K}\Omega \cdot (W_0/W)$$

增加晶体管的宽度能缩小这一时间常数，但这也增加了前一级门的负载。因此，晶体管的尺寸受输入负载电容的限制。

很遗憾，进位链的分布 RC 特性使传播延时与位数 N 成平方关系。为了避免这一点，需要插入信号缓冲反相器。正如在已讨论过的，缓冲器的最佳级数取决于反相器的等效电阻以及传输管的电阻和电容。在我们的工艺中，以及其他大多数实际情形中，这一数目在 3 到 4 之间。增加反相器使总传播延时与 N 成线性关系，这与逐位进位加法器的情形一样。

9.8.3 二进制加法器：逻辑设计考虑

逐位进位加法器只适用于实现较小字长的加法。大多数台式计算机采用 32 位的字长，而服务器则要求 64 位，诸如大型机、超级计算机或多媒体处理器等速度很快的计算机要求的字长达 128 位。加法器延时与位数的线性关系使得采用逐位进位加法器很不实际。因此需要进行逻辑优化，使加法器的 $t_p < O(N)$ 。下面将简短地讨论一些优化方法，我们将集中在电路设计方面，因为这里介绍的大部分结构在传统的逻辑设计文献中是熟知的。

进位旁路加法器（Carry-Bypass Adder）

考虑图 9.33（a）中的 4 位加法器模块。假设 A_k 和 B_k （ $k = 0, \dots, 3$ ）的值使所有的

进位传播信号 P_k 为高电平。在这一状况下，一个进位输入 $C_{i,0} = 1$ 传播通过整个加法器链并使进位输出 $C_{o,3} = 1$ 。换言之：

如果 $(P_0P_1P_2P_3 = 1)$ ，则 $C_{o,3} = C_{i,0}$

否则发生进位消除或进位产生 (9.9)

这一信息可以用来加速加法器的操作，如图 9.33 (b) 所示。当 $BP = P_0P_1P_2P_3 = 1$ 时，进位输入通过旁路晶体管 M_b 立即送至下一个模块，因而称之为进位旁路加法器 (*carry-bypass adder*) 或进位跳越加法器 (*carry-skip adder*)。如果 $P_0P_1P_2P_3 \neq 1$ ，则通过正常路径得到进位输出。

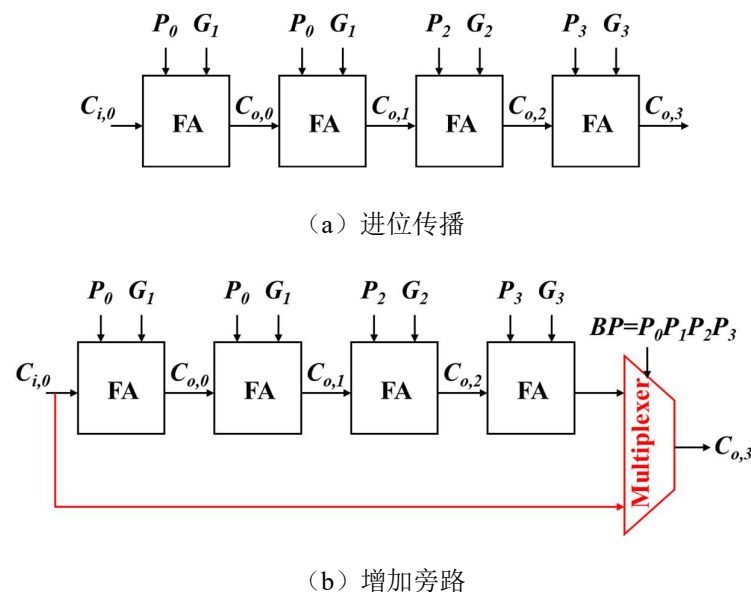


图 9.33 进位旁路结构——基本原理

例 9.4 曼彻斯特进位链加法器中的进位旁路

图 9.34 为用曼彻斯特进位链实现的加法器电路中可能的进位传播路径。图中显示了进位旁路如何加速加法运算：进位通过旁路传播，或者在进位链中的某处产生进位。在这两种情形中，延时都小于通常的逐位进位结构。因增加旁路而增加的面积很小，一般在 10~20% 的范围内。但是，增加旁路破坏了规则的位片结构。

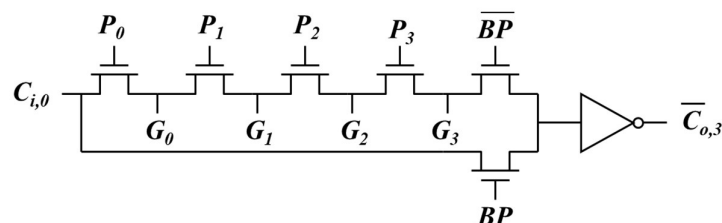


图 9.34 用曼彻斯特进位链实现的旁路加法器

下面我们来计算一个 N 位加法器的延时。首先假设整个加法器被划分成 (N/M) 个等长的旁路级，每一级含有 M 位。由图 9.35 可以推导出这个加法器总传播时间的近似表达式 (9.10)。即，

$$t_{adder} = t_{setup} + Mt_{carry} + (N/M - 1)t_{bypass} + (M - 1)t_{carry} + t_{sum} \quad (9.10)$$

其中各部分参数定义如下：

- t_{setup} ：形成进位产生信号和进位传播信号所需要的固定时间。
- t_{carry} ：通过一位的传播延时。最坏情形下通过具有 M 位的一个级的进位传播延时大约为 t_{carry} 的 M 倍。
- t_{bypass} ：通过一级旁路多路开关的传播延时。
- t_{sum} ：产生最后一级的“和”所需要的时间。

关键路径在图 9.35 的方框图中以灰色表示。由公式 (9.10) 可知 t_{adder} 与位数 N 仍然为线性关系，因为在最坏情况下进位产生于第一位的位置，逐位通过第一个模块，跃过 $(N/M - 2)$ 个旁路级，并且被吸收在最后一位的位置上而不产生输出进位。每一旁路模块的最优位数是由工艺参数来决定的，如旁路选择多路开关的额外延时、在进位链中的缓冲要求以及通过逐位进位和通过旁路的延时的比值。

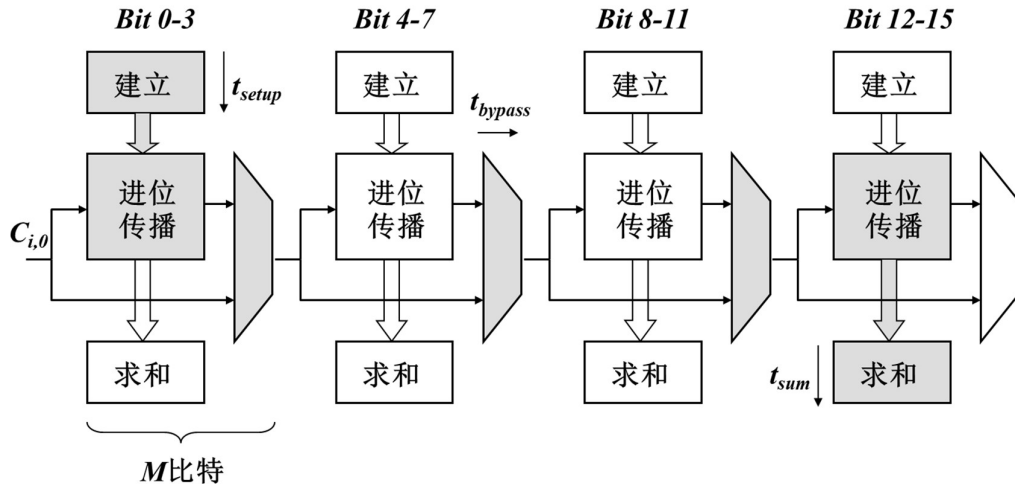


图 9.35 ($N=16$) 进位旁路加法器的组成（灰色部分为最坏情形的延时路径）

尽管进位旁路加法器的延时与位数 N 仍然是线性关系，但旁路加法器延时增加的斜率比在逐位进位加法器中要平缓，如图 9.36 所示。这一差别对于较大的加法器是很明显的。注意，逐位进位加法器实际上在 N 值较小时比较快，这时额外的旁路多路开关的开销使旁路结构并不具有什么意义。交叉点（即何时采用进位旁路）取决于工艺考虑，通常 N 在 4 到 8 位之间。

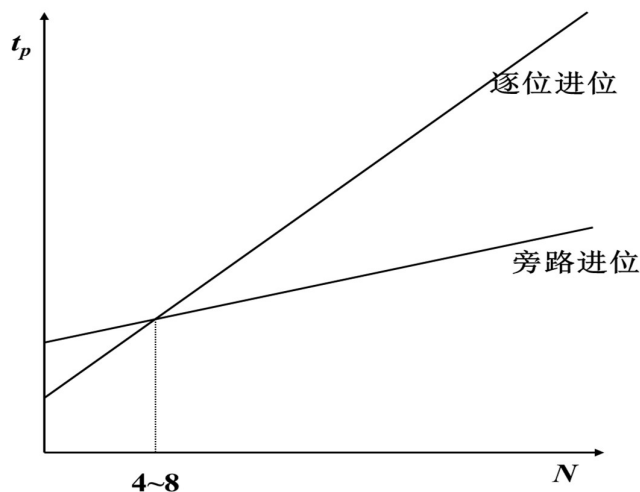


图 9.36 逐位进位加法器与进位旁路加法器传播延时的比较

线性进位选择加法器 (Linear Carry-Select Adder)

在逐位进位加法器中，每一个全加器单元必须等待输入进位到达之后才能产生一个输出进位。避免这一线性依赖关系的一种方法是预先考虑进位输入两种可能的值，并提前计算出针对这两种可能性的结果。一旦输入进位的确切值已知，正确的结果就可以通过一个简单的多路开关级很容易地选出。这一设想的实现非常恰当地称为进位选择加法器 (carry-select adder)，如图 9.37 所示。考虑加法器相加第 k 位至第 $k+3$ 位的模块，它不再等待第 $k-1$ 位的输出进位到达，而是对 0 和 1 两种可能性都进行分析。从电路的角度来看，这意味着实现了两个进位路径。当 $C_{0,k-1}$ 最终确定下来后，就可以通过多路开关选择 0 或者 1 路径的结果，这可以用一个最小的延迟时间来完成。从图 9.37 中可以清楚地看到，进位选择加法器所增加的硬件开销限于一个额外的进位路径和一个多路开关，大约等于逐位进位结构的 30%。

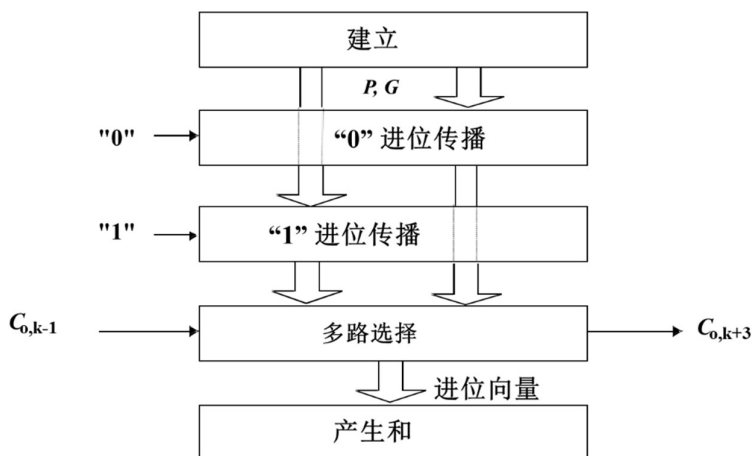


图 9.37 4 位进位选择模块——拓扑结构

一个完整的进位选择加法器现在就可以如进位旁路方法那样通过链接许多相等长度的加法器级来构成（见图 9.38），图中关键路径以灰色显示。通过观察这一电路，可以推导出这一模块最坏情形下传播延时的一阶模型，即可写为：

$$t_{adder} = t_{setup} + Mt_{carry} + \frac{N}{M}t_{mux} + t_{sum} \quad (9.11)$$

式中， t_{setup} 、 t_{sum} 和 t_{mux} 是固定延时， N 和 M 分别代表总位数和每一级的位数， t_{carry} 是进位通过单个全加器单元的延时。通过单个模块的进位延时正比于该级的长度，即等于 Mt_{carry} 。

加法器的传播延时再次线性地正比于 N [公式 (9.11)]。形成这一线性关系的原因是在最坏情形下选择 0 或 1 的运算结果的模块选择信号仍然必须逐一通过所有的级。

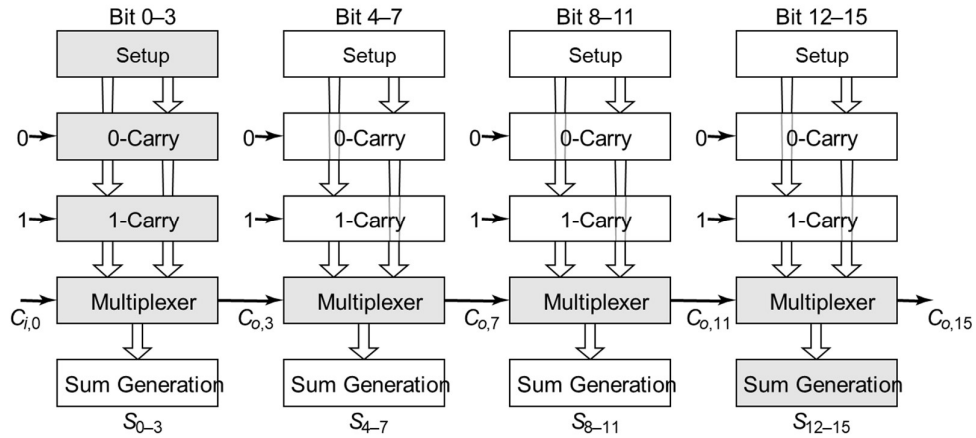


图 9.38 16 位线性进位选择加法器（关键路径以灰色显示）

平方根进位选择加法器（Square-Root Carry-Select Adder）

为了优化设计，最重要的是首先确定关键时序路径的位置。考虑一个 16 位线性进位选择加法器的情形。为了简化讨论，假设全加器和多路开关单元具有相同的传播延时，并都等于一个归一化的值 1。在最坏情形下相对于输入时间的信号到达电路不同节点的时间标注在图 9.39 (a) 上。这一分析显示加法器的关键路径逐级通过后续各级的多路开关电路。

这里出现了一个极好的机会。考虑最后一级加法器的多路开关门。这个多路开关的输入是这模块的两个进位链以及来自前一级模块的多路开关信号。可以观察到这些信号的到达时间有明显的差别，进位链的结果（多路开关的两路数据输入）在多路开关控制信号到达之前很久就已稳定下来。因此使这两条路径的延时相等是有意义的。这可以通过在该加法器中逐渐增加后续各级的位数来实现，因而后续各级需要有更多的时间来产生进位信号。例如，第一级可以相加两位，第二级是 3 位，第三级是 4 位，等等，如图 9.39 (b) 所示。图中标出的到达时间表明这一加法器的拓扑结构虽然增加了额外一级，

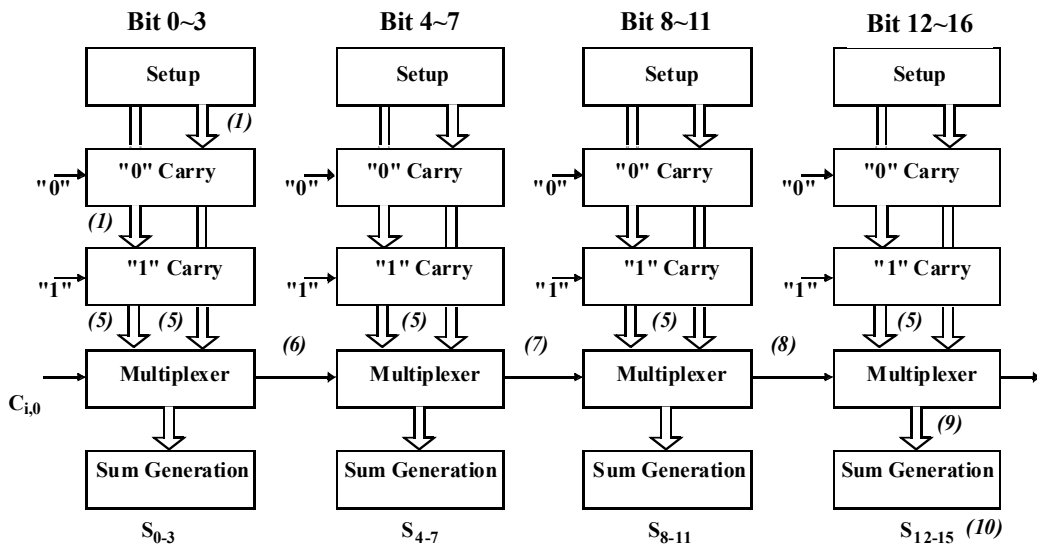
但还是比线性结构快。事实上，一个 20 位加法器的传播延时也和它一样。注意，在各个多路开关节点上两条路径到达时间的差别已消除。

实际上，这一加法器逐级加长的简单方法使加法器结构具有亚线性延时特性。这可以通过以下分析来说明：假设一个 N 位加法器含有 P 级，第一级相加 M 位。后续各级依次增加一位，于是下述关系成立：

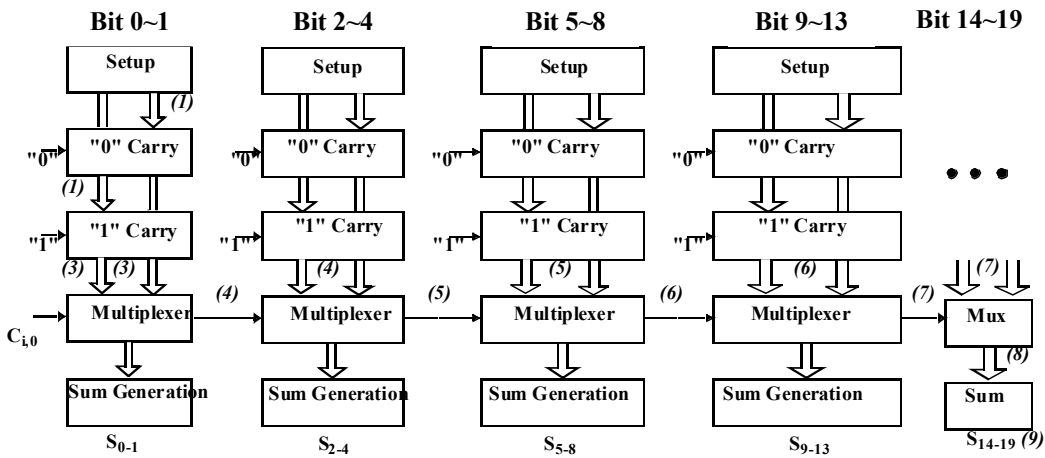
$$N = M + (M+1) + (M+2) + \dots + (M+P-1) = P^2/2 + P(M-1/2) \quad (9.12)$$

如果 $M \ll N$ （例如 $M=2$ ， $N=64$ ），则第一项起主要作用，公式（9.12）可简化为：

$$N \approx P^2/2 \quad \text{或} \quad P = \sqrt{2N} \quad (9.13)$$



(a) 线性结构



(b) 平方根结构

图 9.39 进位选择加法器中最坏情形下信号到达的时间（信号到达时间表示在括号内）

利用公式 (9.13) 重写公式 (9.11)，可以把 t_{adder} 表示成 N 的函数：

$$t_{adder} = t_{setup} + Mt_{carry} + \sqrt{2N}t_{mux} + t_{sum} \quad (9.14)$$

所以，对于大的加法器 ($N \gg M$) 延时与 $\sqrt{N}$ 成正比，即 $t_{adder} = O(\sqrt{N})$ 。这一平方根关系具有重要意义，这可以从图 9.40 看出，图中画出了线性进位选择加法器和平方根进位选择加法器的延时与位数 N 的关系曲线。可以看到，当 N 值很大时， t_{adder} 几乎变为常数。

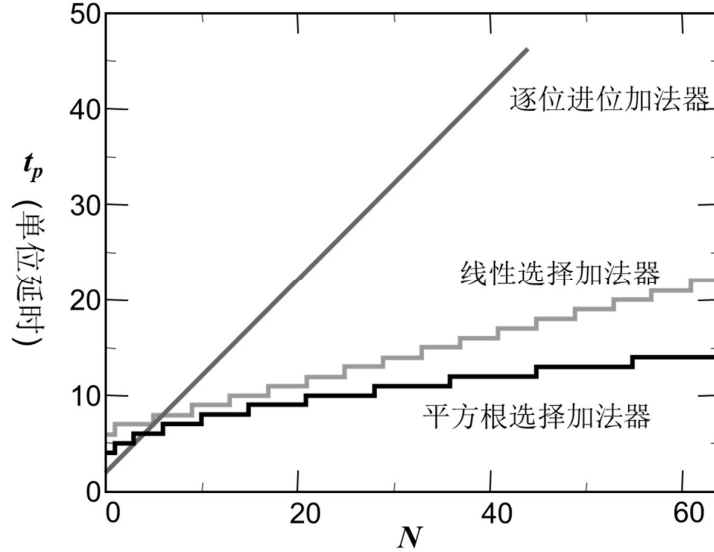


图 9.40 平方根进位选择加法器与逐位进位加法器以及线性选择加法器传播延时的比较
(采用单位延时模型模拟单元延时)

超前进位加法器 (Carry-lookahead Adder)

单一超前进位加法器 在设计更快速的加法器时，避免逐级进位效应至关重要，而这一效应仍然以某一种形式存在于进位旁路和进位选择加法器中。超前进位原理提供了一种有可能解决这一问题的方法。如前所述，在 N 位加法器中每一位的位置上都存在下列关系：

$$C_{o,k} = f(A_k, B_k, C_{o,k-1}) = G_k + P_k C_{o,k-1} \quad (9.15)$$

通过展开 $C_{o,k-1}$ 可以消除 $C_{o,k}$ 对 $C_{o,k-1}$ 的依赖关系：

$$C_{o,k} = G_k + P_k C_{o,k-1} = G_k + P_k (G_{k-1} + P_{k-1} C_{o,k-2}) \quad (9.16)$$

$C_{o,k}$ 的完全展开式为：

$$C_{o,k} = G_k + P_k (G_{k-1} + P_{k-1} (\dots + P_1 (G_0 + P_0 C_{i,0}))) \quad (9.17)$$

其中 $C_{i,0}$ 通常等于零。

这一展开关系可以用来实现一个 N 位的加法器。每一位的进位输出与“和”位输出都与前面的位无关，因而有效地消除了逐位进位效应，因此加法时间应当与位数无关。图 9.41 是一个超前进位加法器整体组成的方块图。

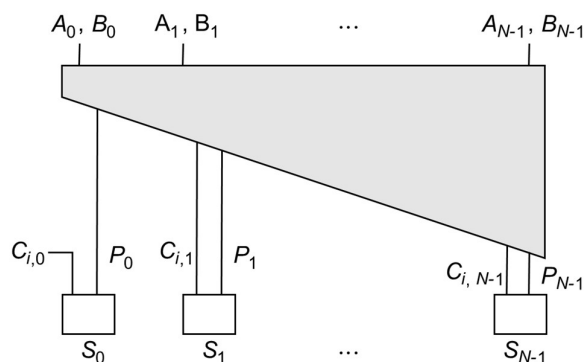


图 9.41 超前进位加法器的原理图

这一高层次的模型包括了某些隐含的依赖关系。当我们研究这一加法器的细节电路图时就会很清楚地发现，与位数无关的加法时间只是一种希望的想法，真正的延时至少随位数呈线性地增加。这一点表现在图 9.42 中，它是 $N=4$ 时公式 (9.17) 的一种可能的电路实现。注意，该电路利用超前进位公式的自对偶性和递归性建立了一镜像结构，与图 9.28 的一位全加器的形式相类似。这一电路的大扇入使它在 N 值较大时极慢。如果用较简单的门来实现，则要求多个逻辑层次。这两种情况都会使传播延时增加。此外，某些信号上的扇出常常过度增加，从而使加法器变得甚至更慢。例如， G_0 和 P_0 信号出现在后续每一位的表达式中。因此在这些线上的电容是很可观的。最后，这一电路实现的面积也随 N 而逐渐增加。因此，这种超前结构只在 N 值较小 (≤ 4) 时有效。

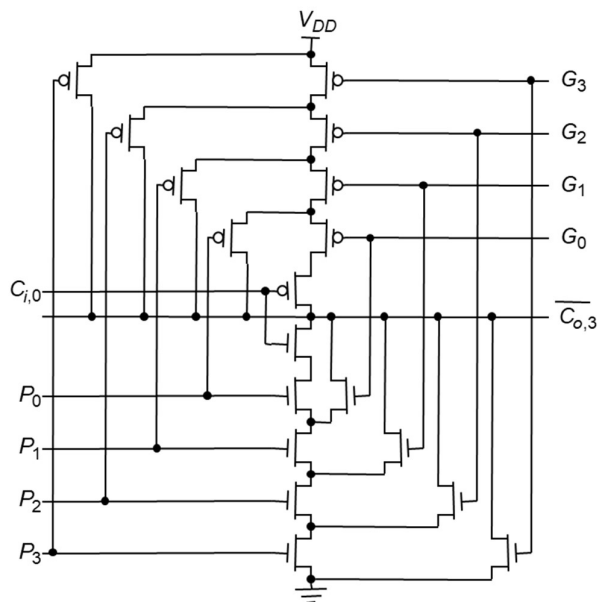


图 9.42 4 位超前进位加法器镜像实现的电路图

对数超前进位加法器——原理 对于一个 N 位的超前进位组，它的晶体管实现具有 $N+1$ 个并行分支并且最多有 $N+1$ 个晶体管堆叠在一起。由于门的分支和晶体管的堆叠较多使性能较差，所以超前进位计算在实际中至多只能限于 2 或 4 位。为了建立非常快速的加法器，需要把进位传播和进位产生组织成递归的树形结构。一个比较有效的实现方法是把进位传播层次化地分解成 N 位的子组合：

$$\begin{aligned}
 C_{o,0} &= G_0 + P_0 C_{i,0} \\
 C_{o,1} &= G_1 + P_1 C_{o,0} = G_1 + P_1 G_0 + P_1 P_0 C_{i,0} = (G_1 + P_1 G_0) + (P_1 P_0) C_{i,0} \\
 &= G_{1:0} + P_{1:0} C_{i,0} \\
 C_{o,2} &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{i,0} = G_2 + P_2 C_{o,1} \\
 C_{o,3} &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{i,0} \\
 &= (G_3 + P_3 G_2) + (P_3 P_2) C_{o,1} = G_{3:2} + P_{3:2} C_{o,1}
 \end{aligned} \tag{9.18}$$

在公式 (9.18) 中，进位传播过程被分解成两位的子组合。 G_{ij} 和 P_{ij} 分别表示一组位（从第 i 位至第 j 位）的进位产生和进位传播函数，因而我们称之为块进位产生和块进位传播信号。如果该组产生一个进位，则 G_{ij} 等于 1，而与输入进位无关。如果一个输入进位传播通过整个这一组，则 P_{ij} 即为 1。这一条件等同于前面讨论过的进位旁路。例如，当进位产生于第 3 位或当进位产生于第 2 位并传播通过第 3 位时，则 $G_{3:2}$ 等于 1（即 $G_{3:2} = G_3 + P_3 G_2$ ）。当输入进位传播通过这两位时， $P_{3:2}$ 为 1（即 $P_{3:2} = P_3 P_2$ ）。

注意，新的进位表达式的形式与原来的一样，只是进位产生和进位传播信号由块进位产生和块进位传播信号所代替。符号 G_{ij} 和 P_{ij} 一般化了原来的进位公式，因为 $G_i = G_{i:i}$ 和 $P_i = P_{i:i}$ 。另一个可能的一般化是把产生和传播函数看做是一对 (G_{ij}, P_{ij}) ，而不是单独的函数。现在可以引入一个新的布尔运算，称为点操作 $(\bullet)$ ，这一操作针对这些成对的函数并且允许对一个块的位进行组合和运算：

$$(G, P) \bullet (G', P') = (G + PG', PP') \tag{9.19}$$

利用这一操作现在我们可以分解 $(G_{3:2}, P_{3:2}) = (G_3, P_3) \bullet (G_2, P_2)$ 。点操作服从结合律，但不符合交换律。

例 9.5 用点操作表示逐位进位加法器

利用点操作，一个 4 位的逐位进位加法器可以重写为：

$$(C_{o,3}, 0) = [(G_3, P_3) \bullet (G_2, P_2) \bullet (G_1, P_1) \bullet (G_0, P_0)] \bullet (C_{i,0}, 0)$$

利用结合律可以重写这一函数，并把 $C_{o,3}$ 表示为两组进位的函数：

$$\begin{aligned}
 (G_{3:0}, P_{3:0}) &= [(G_3, P_3) \bullet (G_2, P_2)] \bullet [(G_1, P_1) \bullet (G_0, P_0)] \\
 &= (G_{3:2}, P_{3:2}) \bullet (G_{1:0}, P_{1:0})
 \end{aligned}$$

利用点操作的结合律，可以构成一个树结构来有效地计算所有 2^i-1 个位置（即 1, 3, 7, 15 等）上的进位（ $i=1, \dots, \log_2(N)$ ）。最重要的优点是在位置 2^i-1 上的进位只需要 $\log_2(N)$ 步就可以计算出来。换言之，一个 N 位加法器的输出进位可以在 $\log_2(N)$ 的时间内计算出来。这比起前面所描述的加法器来有很大的改进。例如，对于一个 64 位的加法器，线性加法器的传播延时正比于 64，对于一个平方根选择加法器，它减小到正比于 8，而对于一个对数加法器，这一比例常数减为 6。这显示在图 9.43 中，图中画出了一个 16 位对数加法器的原理图。位置 15 处的进位通过联合块(0:7)和(8:15)的结果计算出。这两个结果中的每一个又是由各层次组成的。例如，(0:7)是由(0:3)和(4:7)组成的，而(0:3)是由(0:1)和(2:3)组成的，等等。

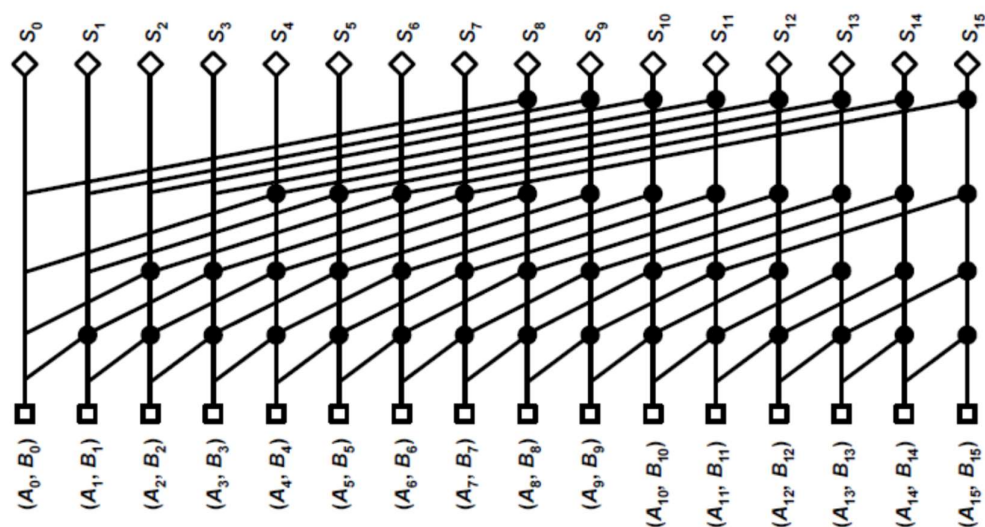


图 9.43 Kogge-Stone 16 位超前进位对数加法器的原理图
(图中□表示建立 P 和 G 信号，●表示点操作，◇表示产生“和”)

只计算在位置 2^i-1 上的进位显然是不够的，还需要推导出在中间位置上的进位信号。实现这一点的一个方法是在每一位上都重复这一树结构，如图 9.43 对于 $N=16$ 的情形所示。例如，位置 6 处的进位通过组合块(6:3)和(2:0)的结果来计算。这整个结构常常称为 *Kogge-stone* 树，它是基数为 2 的树结构中的一种。基数为 2 是指这一结构是二进制树：它在每一结构层次上一次只组合两个进位字。整个加法器需要 49 个复杂逻辑门，每个都实现点操作。此外，还需要 16 个逻辑模块在第一层上产生进位传播和进位产生信号（ P_i 和 G_i ）以及 16 个门产生和。

设计举例——用动态逻辑实现一个超前进位加法器

如果最终目的是实现很高的性能，那么联合超前进位（CLA）技术和动态逻辑似乎是一种理想的方法。因此经历一个动态 CLA 设计的全过程是很有益的。

第一个模块产生进位传播和进位产生信号，如图 9.44 所示。所附加的单独用来驱动保持器的反相器稍难理解一些，这一技术在驱动较大扇出的门中是很有用的。在翻转开

始后通过使保持器的驱动器与该电路的扇出间失去联系可以使保持器很快脱离工作。反之，驱动后续逻辑门的反相器则被优化来驱动两个（对于 G 输出）或三个（对于 P 输出）NMOS 下拉网络的扇出。

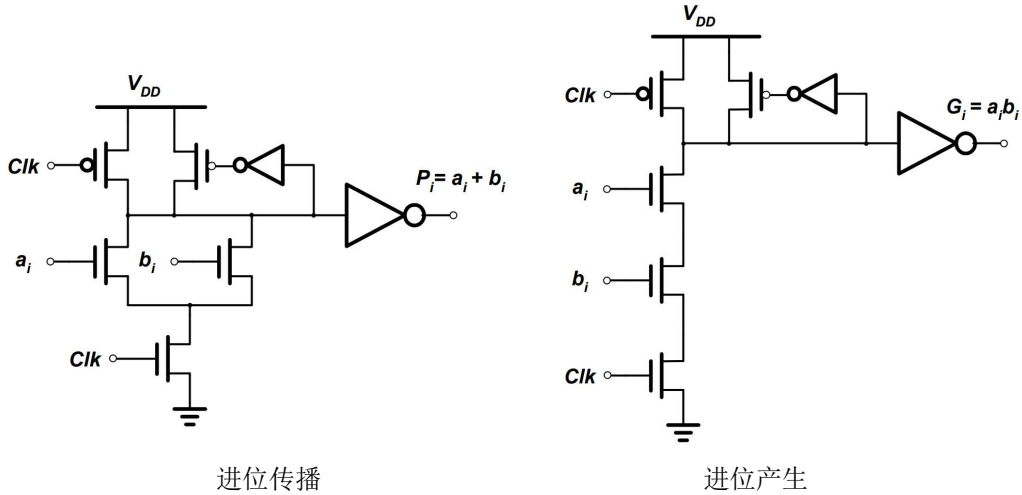


图 9.44 进位传播和进位产生信号的动态实现

图 9.43 中每个黑圆点代表计算块级的进位传播和进位产生信号的两个门，如图 9.45 所示。由于这两个门的位置不在一条流水线的开始处，所以如在第 8 章已讨论过的，求值晶体管（也称为最底下开关）为可选件。这一方法通常用在动态数据通路中。在预充电阶段，多米诺门的所有输出都保证处于低电平，从而关断了在后续多米诺级中的任何放电路径。在除第一级以外的任何一级都取消最底下开关可以减小该门的逻辑努力，而加速求值过程。例如图 9.45 中产生进位传播信号的门的逻辑努力为 $2/3$ 而不是 1 （假设反相器是对称的）。然而这一方法有一个缺点：在预充电阶段，没有最底下开关的所有门中都存在一个短路电流，直到它们的输入被预充电为止。为了避免这一短路电流，每一级的时钟 Clk_k 比它的前一级延迟一点时间。这一方法称为时钟延迟多米诺（*clock-delayed domino*）。注意，对于每一级位片单元时钟都延迟相同的时间，从而简化了电路实现。

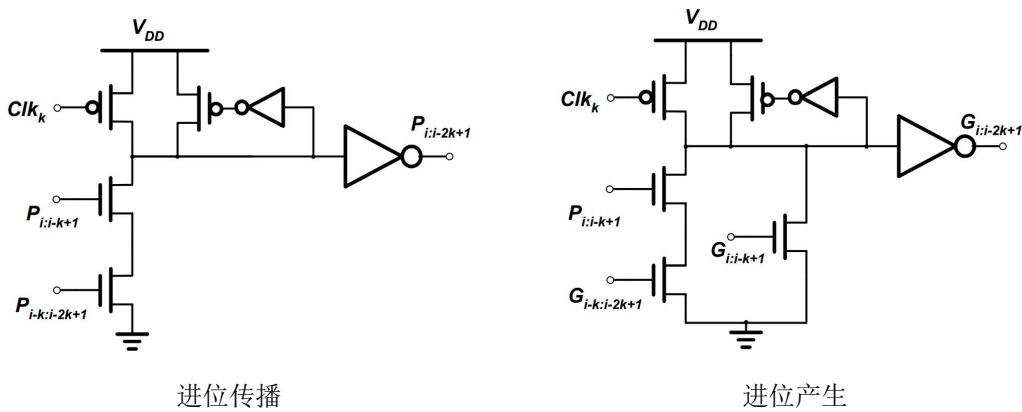


图 9.45 动态逻辑中实现的点操作：在第 k 级第 i 位处

把几个逻辑级按位片式放在一起，即在 P - G 产生之后有 6 个点操作，就可以构成一

个 64 位的加法器。为完成动态加法器设计还缺少的一个辑级是最终和的产生。和的产生需要有 XOR 功能，而它不能很容易地用多米诺逻辑来实现。虽然可以采用静态 XOR 门，但这些门会产生非单向的过渡，因此不能用来驱动其他多米诺门。这本身也许不是一个问题，因为和的产生一般都是加法运算的最后一级。然而在和产生之后的锁存器却不能是透明的，因为这可能会违反后续多米诺级的翻转规则。

用多米诺实现求和的一种方法是通过求和选择 (*sum selection*) 电路，在此电路中求和的两种条件计算为 $S_i^0 = \overline{a_i} \oplus \overline{b_i}$ 及 $S_i^1 = a_i \oplus b_i$ 。于是图 9.46 的动态门就可以根据输入进位选择其中的一种可能性。

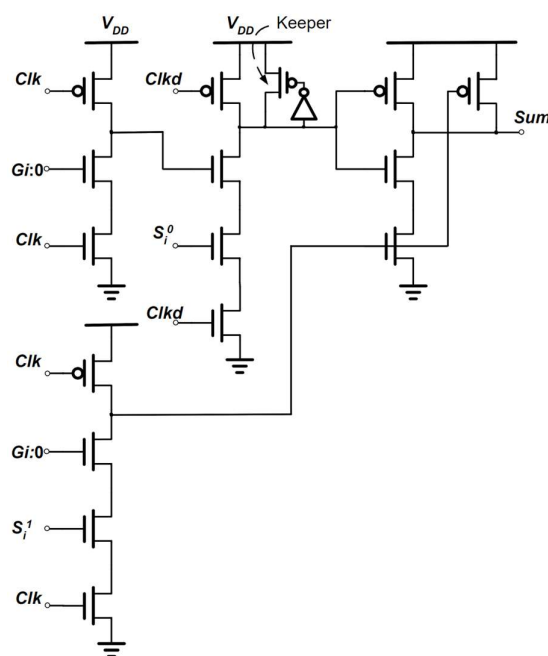


图 9.46 动态逻辑实现和选择电路

多路开关门的实现要求三个逻辑层，这是因为在多米诺逻辑中没有任何互补的进位。与通常一样，在所有动态节点上都应当放有保持器，但图中所示的那个保持器绝对是最重要的。前面两个多米诺级（在左上部）违反了多米诺逻辑的设计规则：两个动态门串联在一起而没有插入反相器。如果这两个门用同一个时钟来求值，那么在第二个门的输出处就有可能发生毛刺 (*glitch*)。使第二个门的时钟 (*Clkd*) 延迟有助于解决这个问题，尽管延迟时钟要求有一个严格的时序界限，即第二个门的所有输入都必须在时钟上升之前完成它们的翻转。

应当强调的是当设计者选择采用这些设计技术时，电路的稳定性不能牺牲。所有的时钟都会出现随机的偏差，但延迟时钟必须具有足够的余量以吸收最坏情形下的时钟偏差。如果 *Clkd* 提前到达，则加法器将会误操作。然而使保持器有合适的尺寸将有助于保证该设计稳定可靠，它在时钟提前到达的情况下可以抑制在第二级输出端的毛刺。保持器的尺寸必须足够大以减少这一虚假翻转，但同时又不能太大以免影响性能。

另一种对数超前进位加法器 快速加法器的设计者在综合考虑面积、功耗或性能时常常回过头来采用其他类型的树结构。我们简短讨论一下另外两种比较常用的结构，即 Brent-Kung 加法器及四进制加法器。

正向二进制树 (forward binary tree) 只实现在 2^N-1 位置上的进位信号:

9-43

正向二进制树结构不足以产生全部进位位，因此需要一个反向二进制树（*inverse binary tree*）来实现其他进位位（在图 9.47 中用灰线表示）。这一结构组合中间的结果以产生其余的进位位。读者可以去验证这一结构确实产生所有进位位的正确表达式。由此得到的结构常称为 **Brent-Kung** 加法器，它采用了 27 个点操作的门，这几乎是完全的二进制树所需要的 49 个点操作门数的一半，并且它所需要的导线也较少。然而布线结构不太规则，并且各个门的扇出也不相同，这使优化性能比较困难。尤其是对于中间节点（ $C_{6,7}$ ）的扇出，在本例中它等于一个求和以及 4 个点操作，是一个应主要考虑的问题，这一事实使 **Brent-Kung** 加法器很不适于非常大（大于 32 位）的加法器。

减少树结构深度的一种选择是在每一结构层次上每次组合四个信号。由此得到的树结构就是四进制树结构，因为它采用的基本构造模块是四线的，如图 9.48 所示。一个 16 位的加法只需要两级进位逻辑。但应当知道这里每一个门是比较复杂的，因此使逻辑级较少并不总能实现较快的操作。

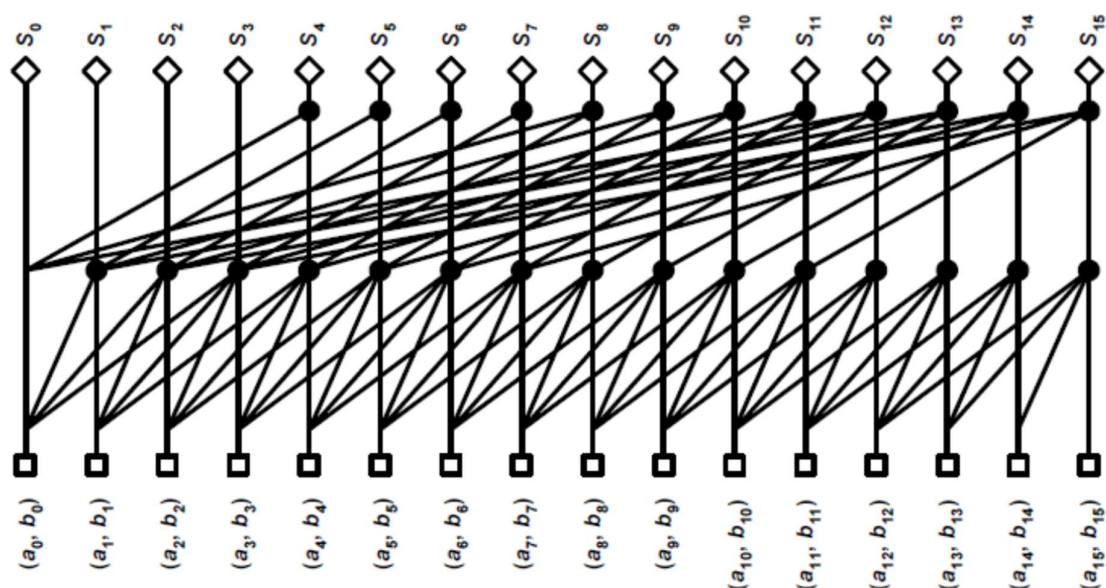


图 9.48 16 位操作数的四进制 Kogge-Stone 树结构

平均而言，一个超前进位加法器要比一个逐位进位加法器大数倍，但对于较大的操作数，它有非常明显的速度优势。对数特性使它在 N 值较大时优于旁路加法器和选择加法器。其交叉点的确切值很大程度上取决于工艺和电路设计。

对于加法器的讨论还没有结束。鉴于它对计算结构性能的重要影响，快速加法器电路的设计已是许多论文的主题。甚至还有可能设计出传播延时与位数无关的加法器结构。这些加法器的例子是进位保留（**carry-save**）结构和冗余二进制算法结构（**redundant binary arithmetic structure**）。这些加法器要求有数字的编码和解码步骤，它们的延时是 N 的函数。因此，它们只有在嵌入到较大的结构（如乘法器或高速信号处理器）中时才有意义。

第九节 十进制加法器

以十进制数直接进行算术运算的计算机或计算器，一般以二进制数形式来表示十进制数。作为这样一台计算机的加法器，必须要有算术电路来接收编码的十进制数，并以相同的编码表示计算结果。对于二进制数加法，只要考虑一对有效位以及前面的进位就足够了。十进制加法器最少需要 9 个输入和 5 个输出，因为每个十进制数需要四位二进制来编码，并且电路必须要有一个输入和一个输出进位。十进制加法器有多种电路，它取决于所代表的十进制数的编码。这里我们主要讨论 BCD 码的十进制加法器。

BCD 加法器

考虑两个十进制数 BCD 码的算术加法，包括来自前一级的进位输入。既然每个输入数不超过 9，输出和就不会大于 $9 + 9 + 1 = 19$ ，其中 1 代表一个输入进位。假设将两个 BCD 码加到一个 4 位二进制加法器上，该加法器就会产生一个二进制和，且计算结果范围为 0 至 19。这些二进制数被列在表 9.8 中，并用符号 K 、 Z_8 、 Z_4 、 Z_2 和 Z_1 来标识。

表 9.8 BCD 加法器的推导

二进制和					BCD 和					十进制数
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

表中 K 代表进位， Z 的下标 8、4、2、1 代表权值，分配给 BCD 码的四个位。二进

制和的那一栏列出了 4 位加法器的输出二进制值。两个十进制数的和输出必须表达成 BCD 码，用列在 BCD 和那一列的形式表示。下面的问题就是要找到一个规则，将二进制和转换成正确的 BCD 和。

从表 9.8 可以看出：当二进制和等于或者小于 1001 时，其对应的 BCD 数是相等的，因而不需要进行转化。当二进制和大于 1001 时，得到的是一个无效的 BCD 表示。解决办法是将二进制数 6（0110）与二进制和数相加，得到正确的 BCD 表示，并产生所需的输出进位。

用于校正的逻辑电路可从表格中得出。当二进制和有一个输出进位 $K=1$ 时就需要校正。其他的从 1010 到 1111 的六种组合也需要进行校正，它们在 Z_8 的位置上都为 1。为了将它们与二进制数 1000 和 1001 加以区分，我们进一步限定 Z_4 或 Z_2 上也必须是 1。校正的条件以及输出进位可以用布尔表达式表示如下：

$$C = K + Z_8Z_4 + Z_8Z_2$$

当 $C=1$ 时，需要将 0110 与二进制和相加并给下一级提供一个输出进位。

两个 BCD 数相加产生 BCD 和的加法器如图 9.49 所示。两个十进制数与输入进位首先加到上面的 4 位加法器上，以产生二进制和。当输出进位等于 0 时，二进制和保持不变。而当其等于 1 时，将二进制数 0110 与下面 4 位加法器产生的和相加。下面加法器产生的输出进位可以被忽略，因为它提供的信息可以在输出进位端得到。一个并行的 n 位十进制加法器需要 n 级 BCD 加法器，其中低一级输出进位必须接到高一级的输入进位上。

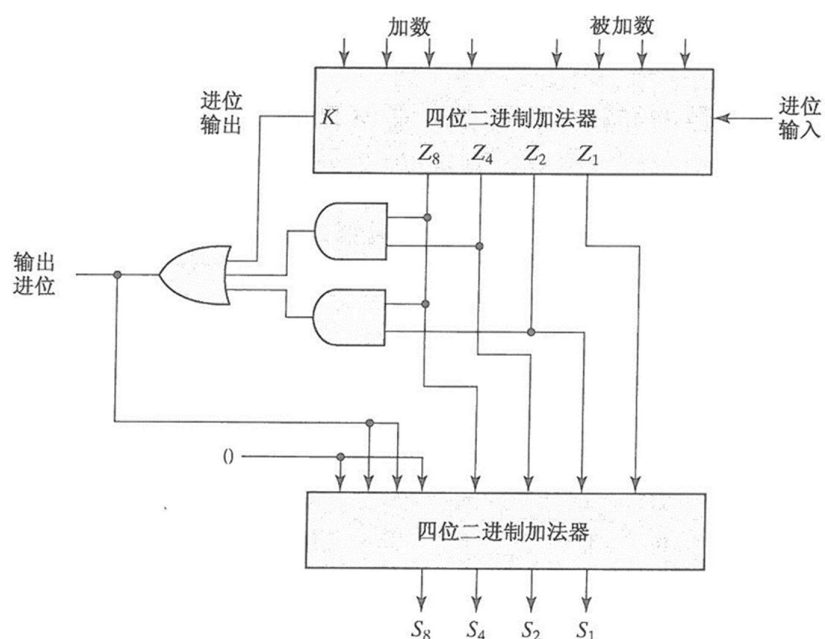


图 9.49 BCD 加法器框图

第十节 二进制减法器

9.10.1 补码

补码在数字计算机中用于简化减法运算和逻辑操作,从而使电路更加简单、价格更加便宜。每个 r 进制系统都有两种类型的补码:补码(*radix complement*)和反码(*diminished radix complement*)。第一种可看作 r 的补码,第二种为 $(r-1)$ 的补码。这里讨论二进制数的 2 的补码和 1 的补码。

反码

给定一个有 n 个数字的 r 进制数 N , 其 $(r-1)$ 的补码定义为 $(r-1) - N$, 即反码。对于二进制数, $r=2$ 和 $r-1=1$, 所以 N 的反码是 $(2^n-1) - N$ 。 2^n 是一个由 1 后面加上 n 个 0 组成的二进制数, 2^n-1 是一个用 n 个 1 表示的二进制数。例如, 如果 $n=4$, 有 $2^4=(10000)_2$ 和 $2^4-1=(1111)_2$ 。因此, 二进制数的反码是用 1 减去其每个数字来得到的。然而, 当用 1 去减二进制数时, 有 $1-0=1$ 或 $1-1=0$, 从而将一位从 0 变为 1 或者从 1 变成 0。因此, 二进制数的反码在形式上就是将 1 改为 0 或将 0 改为 1。如:

1011000 的反码是 0100111

0101101 的反码是 1010010

补码

一个有 n 数字的 r 进制数 N , 其 r 的补码定义为 $r^n - N$ (分 $N \neq 0$ 和 $N=0$ 两种情况)。与前面的反码相比, r 的补码实质就是反码加 1, 即 $r^n - N = [(r^n-1) - N] + 1$ 。因此, 二进制数 101100 的补码是 $010011+1=010100$, 就是将它的反码值加 1 得到的。

2^n 是用一个 1 后面加上 n 个 0 表示的数。因此, 上述二进制数的补码也可以这样来构成, 不改变两个低位的 0 和第一个 1, 再将其他更高位的有效数字用相反的数字代替, 即用 0 代 1, 用 1 代替 0。因此

二进制数 1101100 的补码是 0010100

和

二进制数 0110111 的补码是 1001001

第一个数的补码是通过不改变低两位的 0 以及第一个 1, 而将其他四个高位有效数字取反, 即用 0 代替 1, 用 1 代替 0。第二个数的补码是不改变最低有效位的 1, 而将所有其他位取反。

在前面的定义中, 都是假设那些数没有小数点。如果原始数 N 包含小数点, 应该将小数点临时去掉以计算 r 补码或 $(r-1)$ 反码, 然后再在相同的位置上将小数点恢复。同样值得强调的是, 补码的补码又重新等于它的原码, N 的补码是 $r^n - N$, 其补码的补码是 r^n

$-(r^n - N) = N$ ，刚好等于原码。

补码的减法

利用借位概念的直接减法在小学就学过了。这种方法是当被减数比减数小时就从高位借 1。人们用纸和笔进行减法运算时，这种方法确实很好，而要用硬件实现减法运算时，这种方法的效率就不如使用补码的方法了。

两个有 n 个数字的 r 进制无符号数做减法 $M - N$ 可以按以下步骤来进行：

1. 将被减数 M 加上减数 N 的补码，即 $M + (r^n - N) = M - N + r^n$ 。
2. 如果 $M \geq N$ ，结果的和将产生可以被丢弃的进位，剩下的就是 $M - N$ 的结果。
3. 如果 $M < N$ ，那么和就不会产生进位，等于 $r^n - (N - M)$ ，也就是 $(N - M)$ 的补码。

常见的计算结果是取和的补码并在其前面加上一个负号。

例 9.6 给定两个二进制数 $X = 1010100$ 和 $Y = 1000011$ ，用补码来实现：(a) $X - Y$ ，(b) $Y - X$

$X =$	1010100		$Y =$	1000011
Y 的补码	$= +$	0111101		
和	$=$	10010001		
抛弃末端进位 2^7	$= -$	10000000	X 的补码	$= +$
结果: $X - Y$	$=$	0010001	和	$=$
				1101111
(a)			(b)	

因此， $Y - X = -(1101111 \text{ 的补码}) = -0010001$ 。

无符号数的减法也可以利用 $(r-1)$ 的反码进行。要知道 $(r-1)$ 的反码只比补码小 1，因此，将被减数和减数的反码相加的结果产生一个和数。当有最后进位时，该和数比正确结果少 1，丢弃该最后进位，并将和数加 1。

9.10.2 带符号的二进制数

正整数（包括零）可以表示成无符号数。然而，为了表示负整数，需要表示符号。在普通算术里，一般用“-”号表示负数，用“+”号表示正数。由于硬件限制，计算机必须用二进制数表示任何事物。通常，二进制数最左边的那个位表示其符号，按惯例 0 表示正号，1 表示负号。

要注意，不论是带符号的二进制数还是无符号的二进制数，在计算机中都是用一组比特来表示的，这一点很重要。究竟是无符号数还是带符号数由用户来确定。若二进制数是带符号的，则其最左边的位表示符号，其余位表示数值；而若二进制数被假定是无

符号数，则最左边的位是该数的最高有效位。例如，01001 可以被认为是 9（无符号数）或是+9（带符号数），最左边的那位为 0。11001 作为无符号数时等于 25，而作为有符号数时是-9，因为此时其最左边位置上的 1 表示负号，而其余的四位表示二进制数 9。如果预先知道数的类型，一般就不会混淆。

上一个例子中，那些带符号数的表示方式称为“符号-数值”表示法。在该表示法中，数由数值和符号（+或-）或表示符号的位（0 或 1）两部分组成，这就是普通算术中带符号数的表示。当算术运算在计算机中实现时，使用“符号-补码”系统表示负数是比较方便的。在这个系统中，负数用它的补码表示。“符号-数值”系统通过改变数的符号对其取反，而“符号-补码”系统则是通过求补码将其取反。因为正数最左边的位总是以 0 开始，所以负数的补码总以 1 开始。“符号-补码”系统可以采用反码（1 的补码）或补码（2 的补码），但最常用的是补码。

例如，我们用八位表示二进制数 9。+9 表示为 00001001，即最左边是符号位 0，跟在后面的是等于 9 的二进制序列。注意，所有八位都必须有一个值。因此，在符号位和第一个 1 之间要插入一些 0。尽管只有一种方法来表示+9，而用八位表示-9 却有三种不同的表示方法：

符号-数值表示法： 10001001

符号-反码表示法： 11110110

符号-补码表示法： 11110111

在“符号-数值”表示法中，-9 是通过把+9 最左边位的 0 改为 1 得到的。在“符号-反码”表示法中，-9 是通过把+9 的所有位取反得到的，包括符号位。“符号-补码”表示的 -9 是通过取+9 的二进制补码得到的，也包括符号位。

表 9.9 列出了所有可能的四位带符号二进制数的三种表示法，与此相对应的十进制数也列在其中作为参考。注意，三种表示法中所有正数是完全相同的，且最左边位（最高有效位）为 0。“符号-补码”表示法中 0 只有一种表示，且为正数，而其他两种表示法中既有正 0 也有负 0，这在普通算术中是不会遇到的。所有负数的最高有效位都是 1，据此我们将其与正数区分开来。四位可以表示 16 个二进制数，在“符号-数值”和“符号-反码”表示中，有八个正数和八个负数，包括两个零。在“符号-补码”表示中，有八个正数（其中包括一个零）和八个负数。

“符号-数值”表示一般用于普通算术运算中，但要用于计算机运算却很麻烦，要分开处理符号和数值。因此，计算机通常采用的是“符号-补码”表示。反码存在着一些难点，在算术运算中很少使用。因为将 0 和 1 互换相当于逻辑求反操作，这一点在逻辑运算中很有用。下面讨论带符号的二进制数时只用“符号-补码”来表示负数，相同的步骤可应用于“符号-反码”表示法，只要如同处理无符号数一样包含最后进位就行了。

表 9.9 带符号的二进制数

十 进 制 数	符号 - 补码	符号 - 反码	符号 - 数值
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	—	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
-8	1000	—	—

算术加法

两个数的加法在“符号-数值”表示中遵循普通算术规则。如果两个数的符号相同，我们可以将两个数值相加，而相加之和的符号保持不变。如果两个数的符号不同，就用绝对值大的数减去绝对值小的数，并将绝对值大的那个数的符号赋给运算结果。例如， $(+25) + (-37) = -(37 - 25) = -12$ ，就是用绝对值大的数 37 减去绝对值小的数 25，运算结果使用数 37 的符号。上述步骤需要比较符号和绝对值，然后再进行加或减。该步骤同样也适用于“符号-数值”法所表示的二进制数。不同的是，对于用“符号-补码”表示的二进制数，在进行两个数相加时，并不需要比较符号或相减运算，而只有加法运算。

两个以“符号-补码”形式表示的二进制符号数的加法，就是把这两个数相加，包括它们的符号位。符号位上所产生的进位要丢弃。

数的加法举例如下：

$$\begin{array}{rcl}
 +6 & 00000110 & \\
 +13 & 00001101 & \\
 +19 & 00010011 & \\
 \hline
 +6 & 00000110 & \\
 -13 & 11110011 & \\
 -7 & 11111001 & \\
 \hline
 \end{array}
 \qquad
 \begin{array}{rcl}
 -6 & 11111010 & \\
 +13 & 00001101 & \\
 +7 & 00000111 & \\
 \hline
 -6 & 11111010 & \\
 -13 & 11110011 & \\
 -19 & 11101101 & \\
 \hline
 \end{array}$$

注意，负数必须首先转换为补码形式。如果相加后得到的和是负数，则给出的结果也是补码形式。例如，-7 表示为 11111001。

上述四个例子的每一个所进行的运算都是包括符号位的加法。符号位上所产生的进位都要被丢弃，最后得到的负数结果自动就是 2 的补码形式。

为了得到正确结果，我们必须保证要有足够的位数表示和数。如果两个 n 位的数相加，得到的结果却是 $n+1$ 位的和数，称为产生了溢出。当人们用纸笔进行加法运算时，溢出并不是问题，因为我们没有对纸的宽度做出限制，我们只要再添一个 0 到正数或再添一个 1 到负数作为它们的最高有效位，从而将其扩展成为 $n+1$ 位，然后再进行相加。溢出在计算机中之所以成为问题，是因为计算机保存数的位数（字长）是有限的，结果哪怕超过限定位数 1 位都不行。

用补码形式表示的负数与“符号-数值”法所表示的负数是不同的。为了确定以“符号-补码”形式表示的负数的值，有必要将其转换为正数，这样的形式我们更为熟悉。例如，带符号的二进制数 11111001 是个负数，因为其最左边位上是 1。该二进制负数的补码是 00000111，等于二进制数+7，因此原负数等于-7。

算术减法

当以补码形式表示负数时，两个二进制带符号数的减法是非常简单的，具体步骤如下：

取减数的补码（包括符号位），将其与被减数（包括符号位）相加，丢弃符号位上所产生的进位。

这种方法的产生是因为在算术运算中，如果改变减数的符号，就可以用加法代替减法运算。下面的关系式可以说明：

$$(\pm A) - (+B) = (\pm A) + (-B)$$

$$(\pm A) - (-B) = (\pm A) + (+B)$$

把一个正数变为负数是很容易的，只要取它的 2 的补码就行了。反过来也很容易，因为负数补码的补码可产生等值的正数。我们来看减法 $(-6) - (-13) = +7$ ，用八位二进制数可将其写成 $(11111010 - 11110011)$ 。通过取减数 (-13) 的 2 的补码可以得到 $(+13)$ ，减法变成了加法。在二进制数的算术运算中，就是 $11111010 + 00001101 = 100000111$ ，丢弃最后进位，我们得到正确的答案是 00000111 (+7)。

对于用“符号-补码”法表示的二进制数，如果按照无符号数的基本运算法则进行相同的加法和减法运算，那是没有意义的。因此，计算机只需要普通的硬件电路来处理这两种类型的算术运算。鉴于此，计算机系统中几乎所有的算术单元都采用了“符号-补码”系统。用户或程序员必须要了解的是，加法或减法运算结果的差异将取决于这些数究竟是带符号数还是无符号数。

9.10.3 二进制减法器

如前所述，无符号二进制数的减法可通过补码方式很方便地实现。减法 $A-B$ 可通过将 B 的补码与 A 相加来实现。二进制数补码的实现可以先求其反码，然后在反码的最低有效位上加 1。反码可通过非门实现，1 可以通过输入进位与和相加。

减法 $A-B$ 的实现电路包括一个加法器，并且在每个数据输入 B 与其对应的全加器输入之间都有一个非门。当进行减法运算时，输入进位 C_0 必须要等于 1。这样，该运算就变成了 A 加上 B 的反码，再加 1。对于无符号数，如果 $A \geq B$ 就给出 $A-B$ ；若 $A < B$ ，则得到 $B-A$ 的补码。对于带符号数，假设没有溢出，结果就是 $A-B$ 。

加法和减法运算可包含在一个普通的二进制加法器电路里。可以通过将每个全加器加上一个异或门来实现，得到的四位加减器电路如图 9.50 所示。

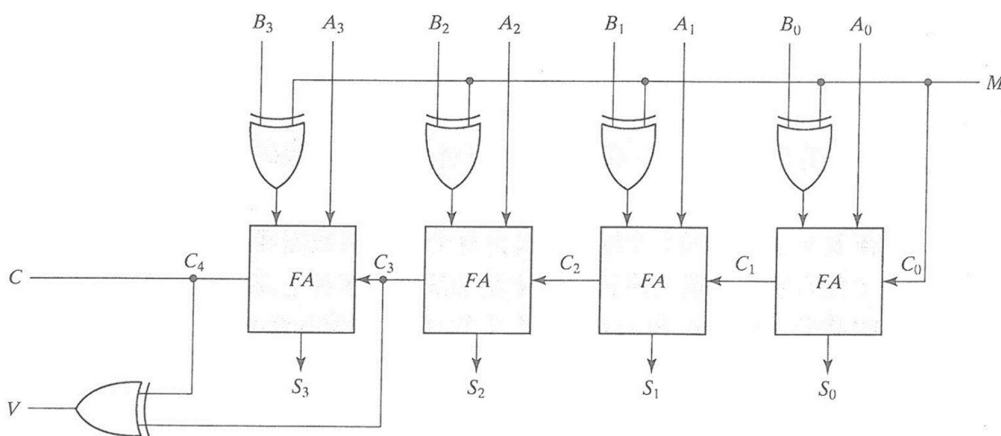


图 9.50 四位加减法器（带溢出检测）

模式输入 M 控制运算类型，当 $M=0$ 时，此电路就是加法器；而当 $M=1$ 时，该电路就成了减法器。每个异或门都收到输入 M 和 B_i ($i=0, \dots, 3$)。当 $M=0$ 时，有 $B \oplus 0 = B$ 。全加器接收到 B 的值，输入进位为 0，电路实现的是 $A+B$ 。当 $M=1$ 时，有 $B \oplus 1 = B'$ ，且 $C_0=1$ ，输入 B 被取反，并且通过输入进位加上 1。该电路实现了 A 加上 B 的补码的运算（输出为 V 的异或门是为了检测溢出）。

溢出

如果两个 n 位数相加的和是 $n+1$ 位，则称为溢出。不管对二进制数还是十进制数，也不管是否有符号，这一点是肯定的。当用纸笔进行加法操作时，溢出不是问题，因为求解时纸的页面大小不受限制。而在数字计算机中，由于计算机字长的限制，无法容纳 $n+1$ 位的数，溢出就成为了问题。正由于这个原因，许多计算机检测溢出的发生。一旦发生溢出，就会有相应提示，使用户能够察觉。

要检测出两个二进制数相加后是否溢出，取决于这些数是带符号数还是无符号数。当两个无符号数相加时，溢出要在最高有效位的最后进位检测。而在带符号数的情况下，

最左边位总是代表符号，且负数用补码表示。当两个带符号数相加时，符号位被当作数的一部分，且最后进位不表示为溢出。

既然一个正数和一个负数相加产生的结果比原先两个数中大的数还要小。因此，若加法运算中有一个数是正数而另一个数为负数，就不会产生溢出。若两个数都为正数或者都为负数，则相加的结果就可能产生溢出。下面我们通过一个例子来看看溢出是如何发生的。假设两个带符号数+70 和+80 存储在两个 8 位寄存器里。每个寄存器能容纳数的范围从二进制的+127 到-128。因为这两个数相加的和是+150，超过了 8 位寄存器容量。如果两个数都为正或都为负，就可能发生这种情况。两个二进制加法及最后的两个进位描述如下：



要注意，本该为正数的 8 位数却有一个负的符号（第 8 位），而本该为负数的 8 位数却有了一个正的符号。然而，如果符号位上的进位作为结果的有效位，则这样获得的 9 位结果才是正确的。因为结果超出了 8 位数，我们就说产生了溢出。

溢出的产生可以由进入到符号位上的进位和符号位上的输出进位来检测。若这两个进位不相等，则产生了溢出。上面例子中的两个进位清楚地说明了这一点。将这两个进位加到一个异或门，若该门的输出等于 1，表明发生了溢出。要使这种方法工作正确，补码必须要通过反码加 1 来计算。这就要注意最大负数求反的情况。

输出为 C 和 V 的二进制加减器如图 9.49 所示。如果把两个二进制数当作无符号数，则 C 位就会检测到一个相加后的进位或相减后的借位。若把这些数当成带符号数，则 V 位就检测溢出。若相加或相减后 $V = 0$ ，就表明没有溢出，且 n 位的结果正确。如果相加或相减后 $V = 1$ ，则运算结果有 $n + 1$ 位，但是该数只有右边的 n 位有效，因此产生了溢出。第 $n + 1$ 位实际上是符号，但被移出了位置。